

Politechnika Warszawska

WYDZIAŁ ELEKTRONIKI
I TECHNIK INFORMACYJNYCH



Instytut Informatyki

Praca dyplomowa inżynierska

na kierunku Informatyka
w specjalności Sztuczna Inteligencja

Optymalizacja inferencji dyfuzyjnego modelu superrozdzielczości
wideo na kartach graficznych o ograniczonej pamięci

Daniel Machniak

Numer albumu 325190

promotor
prof. dr hab. inż. Przemysław Rokita

WARSZAWA 2026

Optymalizacja inferencji dyfuzyjnego modelu superrozdzielczości wideo na kartach graficznych o ograniczonej pamięci

Streszczenie

Modele superrozdzielczości wideo (VSR) oparte na dyfuzji generują bardziej naturalne i ostre obrazy niż klasyczne metody, jednak ich ograniczeniem pozostaje bardzo wysoki koszt obliczeniowy. Choć modele jednokrokowe, takie jak *FlashVSR* [1], eliminują narzut iteracyjnego odszumiania, nadal wymagają ogromnej ilości pamięci. Model ten projektowano z myślą o sprzęcie klasy serwerowej, dlatego jego uruchomienie na standardowych, konsumenckich kartach graficznych jest praktycznie niemożliwe ze względu na deficyt pamięci VRAM.

Celem pracy było dostosowanie modelu *FlashVSR* do działania na karcie graficznej wyposażonej w 10 GB pamięci VRAM, przy zachowaniu akceptowalnej jakości rekonstrukcji i czasu przetwarzania. W tym celu referencyjną implementację przekształcono w modułowy pakiet i wdrożono trzy główne osie optymalizacji: kafelkowanie przestrzenne i czasowe, całkowitoliczbową kwantyzację transformera dyfuzyjnego oraz wymienne jądra mechanizmu uwagi. Wydajność oraz jakość zoptymalizowanego potoku przetestowano na nagraniach z degradacjami syntetycznymi i rzeczywistymi.

Badania wykazały, że kluczowym warunkiem uruchomienia modelu na sprzęcie o ograniczonych zasobach jest kafelkowanie przestrzenne, które jako jedyne uniezależnia zapotrzebowanie na pamięć od rozdzielczości wejściowej. Z kolei wykorzystanie innego wariantu uwagi blokowo-rzadkiej odczuwalnie skróciło czas inferencji. Zastosowanie kwantyzacji pozwoliło na dodatkową redukcję zużycia pamięci, choć odbyło się to kosztem dłuższego czasu przetwarzania. Co istotne, ewentualny spadek jakości wygenerowanego obrazu wynikał niemal w całości z zastosowania kafelkowania, podczas gdy wprowadzone optymalizacje obliczeniowe pozostały praktycznie neutralne dla jakości rekonstrukcji.

Słowa kluczowe: superrozdzielczość wideo, modele dyfuzyjne, FlashVSR, kwantyzacja, mechanizmy uwagi, optymalizacja inferencji

Inference optimization of a diffusion-based video super-resolution model on memory-constrained GPUs

Abstract

Diffusion-based video super-resolution (VSR) models produce sharper and more natural images than conventional methods, but their high computational cost remains a limitation. Although one-step models such as *FlashVSR* [1] eliminate the overhead of iterative denoising, they still require a large amount of memory. The model was designed with server-class hardware in mind, which makes running it on standard consumer graphics cards practically infeasible due to insufficient VRAM.

The aim of this work was to adapt the *FlashVSR* model to run on a graphics card equipped with 10 GB of VRAM, while maintaining acceptable reconstruction quality and processing time. To this end, the reference implementation was refactored into a modular package and three main optimization axes were implemented: spatial and temporal tiling, integer quantization of the diffusion transformer, and interchangeable attention kernels. The performance and quality of the optimized pipeline were evaluated on footage with both synthetic and real-world degradations.

The experiments showed that spatial tiling is the key prerequisite for running the model on resource-constrained hardware, as it is the only technique that makes peak memory usage independent of the input resolution. Replacing the block-sparse attention variant, in turn, noticeably shortened inference time. Quantization allowed a further reduction in memory usage, although at the cost of longer processing. Importantly, the observed decrease in output quality was almost entirely attributable to tiling, whereas the computational optimizations proved practically neutral with respect to reconstruction quality.

Keywords: video super-resolution, diffusion models, FlashVSR, quantization, attention mechanisms, inference optimization

Spis treści

1 Wstęp	1
1.1 Motywacja i sformułowanie problemu	1
1.2 Cel i zakres pracy	1
1.3 Struktura pracy	2
2 Podstawy teoretyczne	3
2.1 Superrozdzielczość wideo	3
2.1.1 Sformułowanie zadania superrozdzielczości wideo	3
2.1.2 Spójność czasowa	4
2.2 Modele dyfuzyjne	4
2.3 Transformer wizyjny	5
2.4 Transformer dyfuzyjny	5
2.5 Optymalizacje mechanizmu uwagi	6
2.5.1 FlashAttention	6
2.5.2 SageAttention	7
2.5.3 Block Sparse Attention	7
2.5.4 SpargeAttention	7
2.6 Kwantyzacja całkowitoliczbowa	7
2.6.1 Mechanizmy mapowania	8
2.6.2 Kwantyzacja wag	8
2.6.3 Kwantyzacja aktywacji	8
2.7 Metryki oceny jakości obrazu i wideo	8
2.7.1 Metryki pełnoreferencyjne	9
2.7.2 Metryki bezreferencyjne	9
3 Przegląd istniejących rozwiązań	11
3.1 Metody konwolucyjne i rekurencyjne	11
3.2 Metody oparte na transformerach	11
3.3 Metody dyfuzyjne	11
3.4 Model FlashVSR	12
3.5 Luka badawcza	13
4 Wybór rozwiązań do implementacji	15
4.1 Mechanizm uwagi	15
4.2 Kwantyzacja	15
4.3 Ograniczenie rozmiaru przetwarzanego fragmentu	16
4.4 Zestawienie technik optymalizacji	16
5 Środowisko i projekt systemu	17
5.1 Platforma sprzętowa	17
5.2 Środowisko programistyczne	17
5.3 Architektura pakietu	17
5.4 Przepływ przetwarzania	18
5.5 Interfejsy systemu	19
5.6 Jakość kodu	19

6 Implementacja	20
6.1 Wymienne warianty mechanizmu uwagi	20
6.2 Integracja kwantyzacji	20
6.3 Kafelkowanie przestrzenne	21
6.4 Kafelkowanie czasowe	21
6.5 Aplikacja pokazowa	21
7 Metodyka badań i testy	24
7.1 Cel badań i pytania badawcze	24
7.2 Zbiory testowe i przygotowanie danych	24
7.3 Metryki	24
7.3.1 Jakość rekonstrukcji	24
7.3.2 Wydajność	25
7.4 Procedura pomiaru wydajności	25
7.5 Plan eksperymentów	26
8 Wyniki i ich omówienie	27
8.1 Wpływ mechanizmu uwagi i kwantyzacji	27
8.2 Wpływ rozmiaru kafla	28
8.3 Granica wykonalności	29
8.4 Jakość rekonstrukcji	30
8.5 Podsumowanie wyników	33
9 Podsumowanie i wnioski	34
9.1 Podsumowanie prac	34
9.2 Wnioski badawcze	34
9.3 Ograniczenia i kierunki rozwoju	35
Bibliografia	36
Spis rysunków	38
Spis tabel	38

1 Wstęp

1.1 Motywacja i sformułowanie problemu

Technologia superrozdzielczości wideo (ang. *Video Super-Resolution*, VSR) służy do poprawy jakości i szczegółowości nagrań o niższej rozdzielczości. Wykorzystuje się ją w wielu praktycznych zastosowaniach, począwszy od rekonstrukcji archiwalnych nagrań i skalowania skompresowanych mediów na platformach streamingowych, aż po poprawę jakości nagrań z monitoringu i badań medycznych.

Podjęcia wykorzystujące głębokie uczenie do VSR od dawna bazują na architekturach splotowych i Transformerach [2], trenowanych z wykorzystaniem funkcji straty na poziomie pikseli. Taka funkcja sprzyja zachowawczym wartościom pikseli, czego skutkiem jest powstawanie zbyt wygładzonych obrazów, pozbawionych detali. Wykorzystanie modeli dyfuzyjnych próbuje rozwiązać te problemy. Jako metody generatywne, odtwarzają one prawdopodobne szczegóły, zamiast uśredniać możliwe rozwiązania. Przekłada się to na ostrzejsze i bardziej naturalne rezultaty. Poprawa ta ma jednak swój koszt, VSR oparte na dyfuzji jest wymagające pod względem obliczeniowym, pamięciowym oraz czasowym. Iteracyjny proces odsumowania i kwadratowa złożoność uwagi w odniesieniu do tokenów czasoprzestrzennych utrudnia praktyczne wykorzystanie.

Częściową odpowiedzią na ten problem są jednokrokowe modele uzyskiwane w wyniku destylacji wiedzy. Reprezentatywnym przykładem jest *FlashVSR* [1]. Twórcy modelu połączyli destylację do modelu jednokrokowego z blokowo-rzadką uwagą, aby zbliżyć się do przetwarzania w czasie rzeczywistym. Destylacja eliminuje koszt wielokrotnego odsumowania, ale nie redukuje narzutu pamięciowego w pojedynczej inferencji, który determinuje, czy model może zostać uruchomiony na danym urządzeniu.

Ten narzut stwarza istotną barierę, ponieważ dostępne modele, takie jak *FlashVSR*, zakładają wykorzystanie akceleratorów klasy serwerowej. Autorzy tego modelu wskazują, że szczytowe zużycie pamięci osiąga 11,13 GB przy 101 klatkach o rozdzielczości wyjściowej 768×1408 [1]. Oznacza to, że sekwencja wejściowa miała wymiary zaledwie 192×352 przy czterokrotnym powiększeniu, czyli parametry znacząco odbiegające od rozdzielczości typowych nagrań. Konsumenckie karty graficzne oferują zazwyczaj od 8 do 12 GB pamięci VRAM; sprzęt referencyjny przyjęty w pracy, czyli NVIDIA RTX 3080, zapewnia 10 GB. Zapotrzebowanie na pamięć przekracza dostępny budżet nawet w tak korzystnym scenariuszu, a deficyt ten rośnie wraz ze wzrostem rozdzielczości wyjściowej. Zasadne staje się zatem pytanie, czy taki model da się zaadaptować do pracy w limicie 10 GB VRAM, oraz ocena, jak wpłynie to na jakość generowanego wideo i czas przetwarzania.

1.2 Cel i zakres pracy

Celem pracy jest umożliwienie inferencji jednokrokowego, dyfuzyjnego modelu superrozdzielczości wideo na karcie graficznej wyposażonej w 10 GB pamięci VRAM, przy zachowaniu akceptowalnej jakości rekonstrukcji i czasu przetwarzania. Jako model bazowy przyjęto *FlashVSR* [1].

Do realizacja tego celu wymagana była refaktoryzacja kodu referencyjnej implementacji do postaci modularnego pakietu oraz zaimplementowanie trzech konfigurowalnych technik redukujących wymagania sprzętowe: wymiennych wariantów mechanizmu uwagi, kwantyzacji, oraz kafelkowania przestrzennego i czasowego. Opracowano również metodykę pomiarową obejmującą czas inferencji, szczytowe zużycie pamięci i jakość rekonstrukcji. Umożliwiło to porównanie uzyskanych konfiguracji między sobą oraz z konfiguracją referencyjną.

W pracy skupiono się na inferencji modelu, bez wykonywania dodatkowego treningu ani dostrajania parametrów, z wykorzystaniem publicznie udostępnionych wag. Głównym wkładem pracy jest adaptacja istniejącego modelu do środowiska o ograniczonych zasobach oraz zbadanie kompromisów między jakością rekonstrukcji, czasem inferencji a zużyciem pamięci.

1.3 Struktura pracy

Praca składa się z dziewięciu rozdziałów. Rozdział drugi wprowadza podstawy teoretyczne: sformułowanie zadania superrozdzielczości wideo, zasadę działania modeli dyfuzyjnych, architektury transformera wizyjnego i dyfuzyjnego, optymalizacje mechanizmu uwagi, kwantyzację całkowitoliczbową oraz metryki oceny jakości. W trzecim rozdziale omówiono istniejące metody VSR, ze szczególnym uwzględnieniem modelu *Flash VSR*. Rozdział czwarty przedstawia kryteria doboru oraz uzasadnienie wyboru trzech osi optymalizacji: mechanizmu uwagi, kwantyzacji i ograniczenia rozmiaru przetwarzanego fragmentu.

Kolejne dwa rozdziały dotyczą części praktycznej. Rozdział piąty opisuje środowisko sprzętowe i programistyczne oraz projekt systemu, a szósty implementację poszczególnych mechanizmów i aplikacji pokazowej.

Rozdział siódmy przedstawia metodykę badań: pytania badawcze, zbiory testowe, metryki, procedurę pomiarową i plan eksperymentów. W rozdziale ósmym zebrano i omówiono uzyskane wyniki, a dziewiąty zawiera podsumowanie, wnioski badawcze oraz ograniczenia i kierunki dalszych prac.

2 Podstawy teoretyczne

2.1 Superrozdzielczość wideo

Superrozdzielczość wideo to zadanie z zakresu widzenia komputerowego, którego celem jest generowanie materiałów wideo o wysokiej rozdzielczości (ang. *high-resolution*, HR) i ulepszonej jakości wizualnej na podstawie wejściowych sekwencji o niskiej rozdzielczości (ang. *low-resolution*, LR). Podstawową różnicą między VSR a superrozdzielczością obrazów (ang. *Single Image Super-Resolution*, SISR) jest obecność wymiaru czasowego w wideo. SISR bazuje wyłącznie na informacji przestrzennej z pojedynczego obrazu, natomiast VSR wykorzystuje dodatkowo silne korelacje między kolejnymi klatkami. Bezpośrednie zastosowanie metod SISR do poszczególnych klatek wideo nie pozwala na uchwycenie zależności czasowych [3].

2.1.1 Sformułowanie zadania superrozdzielczości wideo

Sformułowanie zadania VSR opiera się na założeniu, że każde nagranie jest niedoskonałym zapisem sceny o nieograniczonej szczegółowości. Z powodu fizycznych ograniczeń sprzętu, kompresji oraz rozmycia ruchu, część oryginalnych informacji zostaje bezpowrotnie utracona. Wideo niskiej rozdzielczości (LR) traktuje się więc jako uproszczoną wersję idealnej sekwencji (HR).

Ogólny model degradacji dla i -tej klatki wideo można zapisać jako funkcję zależną od klatki docelowej I_i^{HR} oraz jej sąsiedztwa czasowego [4]:

$$I_i^{\text{LR}} = \phi\left(I_i^{\text{HR}}, \{I_j^{\text{HR}}\}_{j=i-N}^{i+N}; \theta_\alpha\right) \quad (1)$$

gdzie I_i^{LR} oznacza obserwowaną klatkę o niskiej rozdzielczości, N oznacza promień czasowy (zakres sąsiednich klatek), a θ_α reprezentuje parametry procesu degradacji (np. szum, rozmycie).

Zadaniem modelu VSR jest odwrócenie tego przekształcenia, czyli znalezienie funkcji odwzorowującej f_{VSR} , sparametryzowanej przez wagi $\theta_{f_{\text{VSR}}}$, która estymuje klatkę wysokiej rozdzielczości $\hat{I}_i^{\text{SR}}$ na podstawie sekwencji klatek wejściowych niskiej rozdzielczości [4]:

$$\hat{I}_i^{\text{SR}} = f_{\text{VSR}}\left(I_i^{\text{LR}}, \{I_j^{\text{LR}}\}_{j=i-N}^{i+N}; \theta_{f_{\text{VSR}}}\right) \quad (2)$$

Odwrócenie to nie ma jednoznacznego rozwiązania. Degradacja bezpowrotnie usuwa część informacji i nie jest odwracalna, ponieważ jednej sekwencji LR odpowiada wiele różnych sekwencji HR. Model nie odtwarza więc oryginału, lecz wskazuje jedno z potencjalnych rozwiązań.

Powyższe klasyczne sformułowania zakładają uproszczone, deterministyczne zniekształcenia, które nie odzwierciedlają warunków rzeczywistych. W praktyce proces utraty jakości wymaga zaawansowanych modeli degradacji, które łączą losowe rozmycia, szum oraz zmiany rozmiaru i artefakty kompresji [5].

2.1.2 Spójność czasowa

Spójność czasowa w kontekście superrozdzielczości wideo polega na utrzymaniu ciągłości, płynności oraz braku zauważalnych zniekształceń i migotania między kolejnymi klatkami wygenerowanej sekwencji HR [3], [4]. Bezpośrednie zastosowanie metod SISR do poszczególnych klatek pomija zależności czasowe, co prowadzi do niestabilności detali i artefaktów wizualnych. W modelach głębokiego uczenia spójność tę realizuje się poprzez techniki wyrównywania klatek i kompensacji ruchu (ang. *motion estimation and motion compensation*, MEMC), konwolucje 3D, moduły rekurencyjne propagujące kontekst czasowy oraz mechanizmy uwagi [3]. Choć metody te poprawiają jakość generowanych sekwencji, analiza wielu klatek znacznie zwiększa złożoność obliczeniową i pamięciową oraz stwarza ryzyko akumulacji i przenoszenia błędów między klatkami [5].

2.2 Modele dyfuzyjne

Modele dyfuzyjne (ang. *Denoising Diffusion Probabilistic Models*, DDPMs) to klasa modeli generatywnych wykorzystująca dwa podstawowe łańcuchy Markova: w przód (ang. *forward chain*), który stopniowo zniekształca dane poprzez dodawanie szumu, oraz w tył (ang. *reverse chain*), przekształcający szum z powrotem w ustrukturyzowane dane [6].

Proces w przód, nazywany również procesem dyfuzji, stopniowo degraduje oryginalne dane x_0 poprzez dodawanie szumu gaussowskiego w ciągu T kroków czasowych. W każdym kroku przejście nakłada szum zgodnie z określonym harmonogramem wariancji β_t , co matematycznie opisuje rozkład warunkowy [7]:

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t\mathbf{I}) \quad (3)$$

Proces ten, stopniowo wprowadzając szum, doprowadza do całkowitej utraty pierwotnej struktury danych, przekształcając ich rozkład w standardowy rozkład Gaussa. Istotną zaletą tego procesu jest możliwość wyrażenia go w postaci zamkniętej. Oznacza to, że można bezpośrednio wygenerować próbkę x_t w dowolnym kroku czasowym t , bez konieczności obliczania kroków pośrednich [6], [7].

Proces w tył (odszumianie) jest procesem odwrotnym, odpowiedzialnym za generowanie nowych danych. Rozpoczyna się od próbkowania losowego wektora szumu ze standardowego rozkładu Gaussa, a następnie stopniowego usuwania szumu poprzez realizację wyuczonego łańcucha Markowa wstecz od kroku T do kroku 1. Ponieważ proces w przód w każdym kroku wprowadza niewielkie ilości szumu, przejścia odwrotne również można modelować jako warunkowe rozkłady Gaussa. Te wyuczalne przejścia są parametryzowane przez głębokie sieci neuronowe i opisywane równaniem [6], [7]:

$$p_{\theta}(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_{\theta}(x_t, t), \Sigma_{\theta}(x_t, t)) \quad (4)$$

gdzie sieć estymuje wartość średnią oraz wariancję w celu odtworzenia obrazu o mniejszym poziomie szumu. W praktyce, zamiast bezpośredniej estymacji parametrów rozkładu μ_{θ} oraz Σ_{θ} , sieć jest najczęściej trenowana do przewidywania szumu gaussowskiego, który został dodany do czystych danych w kroku t .

W klasycznych modelach dyfuzyjnych reprezentacje pośrednie mają ten sam wymiar co dane wejściowe. Aby zniwelować wynikające z tego ograniczenia obliczeniowe, stosuje się dyfuzję w przestrzeni ukrytej (ang. *Latent Diffusion Models*, LDMs) [6]. Podejście to opiera się na hipotezie rozmaitości (ang. *manifold hypothesis*), zakładającej, że naturalne sygnały leżą w podprzestrzeniach o znacznie niższym wymiarze. Modele LDM wykorzystują autoenkoder do skompresowania wysokowymiarowych danych w ciągłą reprezentację ukrytą (ang. *latent space*). Właściwy proces dyfuzji odbywa się wyłącznie w tej skompresowanej przestrzeni, a dekodery mapuje odszumiony wektor ukryty z powrotem do przestrzeni pikseli, co znacząco redukuje zapotrzebowanie na pamięć i moc obliczeniową [6].

2.3 Transformer wizyjny

Architektura Transformer [2] znalazła skuteczne zastosowanie również w wizji komputerowej jako Transformer wizyjny (ang. *Vision Transformer*, ViT) [8]. Zamiast wykorzystywać filtry splotowe, ViT dzieli obraz na płyty (**patches**) i traktuje je analogicznie do tokenów słów w modelach przetwarzania języka naturalnego.

Dane wejściowe $x \in \mathbb{R}^{H \times W \times C}$ są dzielone na sekwencję płyt $x \in \mathbb{R}^{N \times (P^2 \cdot C)}$, gdzie (H, W) to rozdzielczość obrazu, C to liczba kanałów, P to rozmiar płyty, a $N = \frac{HW}{P^2}$ to wyjściowa liczba płyt stanowiąca długość sekwencji wejściowej dla Transformera. Następnie każdy płyt jest rzutowany liniowo do wymiaru ukrytego D , a do uzyskanych wektorów dodawane są wyuczalne osadzenia pozycyjne w celu zachowania informacji o strukturze przestrzennej. Tak przygotowane osadzenia trafiają do enkodera Transformera. Po przetworzeniu sekwencji konieczne jest jej ponowne odwzorowanie do przestrzeni pikseli. Zazwyczaj wykorzystuje się do tego konwolucję podpikselową (ang. *pixel shuffle*). Metoda ta reorganizuje elementy wyjściowych map cech poprzez przekształcenie głębokości w wymiary przestrzenne (ang. *depth-to-space*), pozwalając na odtworzenie wysokiej rozdzielczości bez wprowadzania dodatkowych trenowalnych parametrów [3].

Adaptacja ViT do superrozdzielczości wideo polega na rozszerzeniu podziału na płyty do przestrzeni trójwymiarowej, co umożliwia jednoczesne modelowanie relacji przestrzennych i czasowych. Poprzez zastosowanie mechanizmu samouwagi (ang. *Self-Attention*), modele VSR dynamicznie szacują istotność różnych klatek wewnątrz analizowanego okna czasowego, co pozwala wyselekcjonować najbardziej informatywne fragmenty z sąsiednich klatek i wykorzystać je do precyzyjnej rekonstrukcji detali [3].

2.4 Transformer dyfuzyjny

Połączenie zalet modeli dyfuzyjnych w przestrzeni ukrytej (LDM) oraz skalowalności architektury ViT doprowadziło do powstania transformerów dyfuzyjnych (ang. *Diffusion Transformer*, DiT) [9]. W przeciwieństwie do klasycznych modeli LDM, które wykorzystują splotowe sieci typu U-Net jako kręgosłup (ang. *backbone*) procesu odszumiania, architektura DiT zastępuje je enkoderem opartym na Transformerze.

Przetwarzanie w DiT rozpoczyna się od skompresowania obrazu do przestrzeni ukrytej za pomocą wstępnie wytrenowanego autoenkodera (np. autoenkodera wariancyjnego, VAE) o zamrożonych wagach. Na tak uzyskanej reprezentacji ukrytej z_0 aplikowany jest proces

dyfuzji, a zaszumiony wektor ukryty z_t dzielony jest na siatkę płatów. Następnie są one rzutowane liniowo na sekwencję tokenów i wzbogacane o sinusoidalne osadzenia pozycyjne. Kluczową innowacją architektury DiT jest sposób wstrzykiwania informacji warunkujących, takich jak krok czasowy t czy etykieta klasy c . Do tego celu wykorzystuje się zmodyfikowane bloki z adaptacyjną normalizacją warstwową (ang. *adaptive layer normalization*, adaLN-Zero). Zamiast standardowej normalizacji, sieć estymuje parametry skali i przesunięcia (γ, β) oraz skalar przeskalowujący α bezpośrednio z sumy wektorów osadzeń warunków. Po przejściu przez bloki DiT, wyjściowa sekwencja tokenów jest dekodowana liniowo i reorganizowana do pierwotnego kształtu reprezentacji ukrytej, skąd dekodер przekształca ją z powrotem do przestrzeni pikseli [9].

Głównym wyzwaniem w adaptacji architektur DiT do zadań superrozdzielczości wideo jest potrzeba uwarunkowania procesu generatywnego nagraniem LR. W modelach VSR opartych na DiT (takich jak *FlashVSR*, omawiany w podrozdziale 3.4) sekwencja HR nie jest generowana z czystego szumu; zamiast tego do sterowania odtwarzaniem detali wykorzystuje się wideo LR. Wprowadzanie informacji warunkującej realizuje się m.in. poprzez rzutowanie klatek LR za pomocą lekkiej warstwy konwolucyjnej (*Proj-In*) i dodanie tak uzyskanych cech do tokenów jako osadzenia warunkujące [1].

2.5 Optymalizacje mechanizmu uwagi

Standardowy mechanizm uwagi oparty na iloczynie skalarnym ze skalowaniem (ang. *Scaled Dot-Product Attention*) stanowi fundament architektury Transformer. Odzworowuje on zapytania (Q), klucze (K) oraz wartości (V) na wektor wyjściowy zgodnie z zależnością [2]:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (5)$$

gdzie d_k oznacza wymiarowość kluczy, a dzielenie przez $\sqrt{d_k}$ zapobiega zanikaniu gradientów dla dużych wartości iloczynu skalarnego. Mimo że operacja ta opiera się na wydajnych mnożeniach macierzowych, charakteryzuje się kwadratową złożonością obliczeniową i pamięciową $O(N^2)$ względem długości sekwencji N . Wynika to z konieczności bezpośredniego zapisywania pełnej, pośredniej macierzy uwagi w pamięci karty graficznej [10].

2.5.1 FlashAttention

FlashAttention [10] to metoda uwagi gęstej, która niweluje ograniczenia pamięciowe poprzez algorytm optymalizujący przepływ danych, nie wprowadzając przy tym żadnych aproksymacji. Zamiast zapisywać pośrednią macierz uwagi w wolniejszej pamięci HBM (ang. *High Bandwidth Memory*), algorytm stosuje technikę kafelkowania (ang. *tiling*). Bloki macierzy Q , K i V są ładowane do znacznie szybszej pamięci podręcznej SRAM na chipie GPU, gdzie stopniowo obliczana jest redukcja *softmax*, a do pamięci HBM zapisywany jest jedynie ostateczny wynik.

2.5.2 SageAttention

SageAttention [11] przyspiesza inferencję uwagi gęstej dzięki zastosowaniu optymalizacji opartej na kwantyzacji. Metoda ta redukuje macierze Q i K do formatu INT8, co pozwala wykorzystać znacznie szybsze operacje mnożenia na nowoczesnych układach GPU w porównaniu z formatami FP16 czy FP8. Macierze P (macierz wag uwagi po operacji softmax) oraz V pozostawiono w precyzji FP16. Aby zapobiec utracie dokładności wynikającej z ograniczonego zakresu liczbowego formatu całkowitoliczbowego, zastosowano technikę wygładzania usuwającą wartości odstające z macierzy K .

Wersja *SageAttention2* [12] wprowadza jeszcze szybszą kwantyzację INT4 dla Q i K oraz kwantyzację FP8 dla P i V , łącząc to z kompleksowym wygładzaniem wartości odstających dla obu macierzy Q i K .

2.5.3 Block Sparse Attention

Block Sparse Attention to blokowo-rzadka metoda aproksymacyjna redukująca złożoność poprzez odrzucanie nadmiarowych obliczeń. Zamiast pełnych interakcji uwaga ograniczana jest wyłącznie do wybranych, niezerowych bloków.

Przykładowo, w modelach takich jak *FlashVSR*, algorytm najpierw wyznacza przybliżoną mapę uwagi poprzez uśrednianie (ang. *average pooling*) cech kluczy i wartości w celu uzyskania reprezentacji blokowych. Następnie wybierana jest grupa k najbardziej istotnych par bloków (ang. *top-k*), do których jest aplikowany pełny mechanizm uwagi. Integracja tej metody z *FlashAttention* pozwala obniżyć złożoność operacji wejścia/wyjścia proporcjonalnie do współczynnika rzadkości [10].

2.5.4 SpargeAttention

SparseAttention [13] to uniwersalna blokowo-rzadka metoda niewymagająca ponownego trenowania, która łączy cechy uwagi rzadkiej oraz kwantyzacji w celu przyspieszenia predykcji bez utraty jakości modelu. Algorytm ten rozszerza metodę *SageAttention* o dwuetapowy filtr, umożliwiający pominięcie zbędnych mnożeń macierzowych. W pierwszym etapie bloki Q oraz K wykazujące wysokie podobieństwo są kompresowane do pojedynczych tokenów, co pozwala na szybsze i bardziej precyzyjne przewidywanie oraz maskowanie rzadkich obszarów na mapie uwagi. W drugim etapie algorytm *sparse warp online softmax* pomija obliczanie iloczynu PV , jeśli lokalna wartość maksymalna w danym bloku jest znacząco mniejsza od wartości maksymalnej w skali globalnej, skutecznie ignorując mało znaczące wartości.

2.6 Kwantyzacja całkowitoliczbowa

Kwantyzacja całkowitoliczbowa (ang. *Integer Quantization*) polega na konwersji wartości o wysokiej precyzji (np. FP32) na nisko-precyzyjne formaty całkowitoliczbowe (np. INT8), co przyspiesza inferencję poprzez wykorzystanie dedykowanych jednostek obliczeniowych i redukcję zapotrzebowania na pamięć. Proces ten opiera się na operacji kwantyzacji, czyli mapowaniu wartości rzeczywistych na całkowite, oraz dekwantyzacji, odpowiedzialnej za odtworzenie wartości przybliżonych [14].

2.6.1 Mechanizmy mapowania

Jednostajna kwantyzacja przekształca rzeczywiste wartości $x \in [\beta, \alpha]$ na b -bitowe reprezentacje całkowitoliczbowe x_q . Przekształcenie to opiera się na transformacji afinicznej (ang. *affine quantization*) $f(x) = s \cdot x + z$ lub jej szczególnym przypadku - transformacji skalowanej (ang. *scale quantization*) $f(x) = s \cdot x$ [14].

Kwantyzacja afiniczna mapuje zakres rzeczywisty na przedział ze znakiem $[-2^{b-1}, 2^{b-1} - 1]$ przy użyciu skali $s = \frac{2^b - 1}{\alpha - \beta}$ oraz całkowitoliczbowego punktu zerowego $z = -\text{round}(\beta \cdot s) - 2^{b-1}$, odpowiedzialnego za dokładne odwzorowanie wartości zero. Samą kwantyzację oraz dekwantyzację definiuje się wzorami:

$$x_q = \text{clip}(\text{round}(s \cdot x + z), -2^{b-1}, 2^{b-1} - 1), \quad \hat{x} = \frac{1}{s}(x_q - z) \quad (6)$$

Kwantyzacja skalowana (w wariancie symetrycznym) zakłada symetrię przedziału wejściowego $[-\alpha, \alpha]$ oraz zakresu bitowego wokół zera (dla INT8 jest to $[-127, 127]$, bez wartości -128). Przy skali $s = \frac{2^{b-1} - 1}{\alpha}$, operacje przyjmują postać:

$$x_q = \text{clip}(\text{round}(s \cdot x), -2^{b-1} + 1, 2^{b-1} - 1), \quad \hat{x} = \frac{1}{s}x_q \quad (7)$$

2.6.2 Kwantyzacja wag

Ponieważ wagi podczas inferencji są statyczne, wyznaczanie ich parametrów kwantyzacji odbywa się w trybie offline. W tym celu stosuje się kwantyzację kanałową (ang. *per-channel*), która zachowuje dokładność modelu przy zróżnicowanych rozkładach cech pomiędzy kanałami. Zakres wartości α i β określa się zazwyczaj za pomocą kalibracji bazującej na maksymalnej wartości (*max calibration*), przy czym zdecydowanie preferuje się wariant symetryczny, ponieważ brak współczynnika z eliminuje dodatkowy narzut obliczeniowy podczas mnożenia macierzy [14].

2.6.3 Kwantyzacja aktywacji

Dynamiczna natura aktywacji wymusza odmienne podejście do wyznaczania zakresów mapowania. Aby umożliwić wydajne mnożenie macierzy, aktywacje kwantyzuje się na poziomie całego tensora (*per-tensor*). Ze względu na częstą obecność wartości odstających, zamiast kalibracji maksymalnej stosuje się kalibrację entropijną lub percentylową, co zapobiega skupieniu większości poprawnych wartości w zbyt wąskim przedziale bitowym.

Połączenie symetrycznej kwantyzacji *per-channel* dla wag oraz kwantyzacji *per-tensor* z kalibracją entropijną lub percentylową dla aktywacji pozwala na prowadzenie obliczeń w pełni na arytmetyce INT8 przy dokładności zbliżonej do modelu bazowego FP32 [14].

2.7 Metryki oceny jakości obrazu i wideo

W celu przeprowadzenia kompleksowej ewaluacji jakości wygenerowanych sekwencji wideo zastosowano zestaw siedmiu metryk badawczych, obejmujący zarówno tradycyjne wskaźniki pełnoreferencyjne (ang. *Full-Reference*), jak i zaawansowane metryki bezreferencyjne (ang. *No-Reference*).

2.7.1 Metryki pełnoreferencyjne

Metryki pełnoreferencyjne wymagają bezpośredniego porównania wygenerowanej klatki lub sekwencji ze źródłowym materiałem referencyjnym (ang. *Ground Truth*, GT).

Stosunek sygnału szczytowego do szumu (PSNR, ang. *Peak Signal-to-Noise Ratio*) [15] jest obiektywną metryką wyznaczającą stosunek maksymalnej możliwej wartości sygnału do błędu średniokwadratowego na poziomie poszczególnych pikseli. Mimo powszechnego stosowania w literaturze, metryka ta zakłada niezależność pikseli i wykazuje niską korelację z ludzką percepcją jakości wizualnej [3].

Wskaźnik podobieństwa strukturalnego (SSIM, ang. *Structural Similarity Index Measure*) [16] ocenia podobieństwo strukturalne między obrazami na podstawie analizy trzech komponentów: jasności (luminancji), kontrastu oraz informacji strukturalnej. Wartości metryki zawierają się w przedziale $[-1, 1]$, gdzie wartość 1 oznacza idealną zgodność. Choć SSIM lepiej odzwierciedla percepcję wzrokową niż PSNR, może dawać sprzeczne z intuicją wyniki w sytuacjach znacznych różnic kolorystycznych lub w bardzo ciemnych scenach [3].

Głęboka percepcyjna miara podobieństwa fragmentów obrazu (LPIPS, ang. *Learned Perceptual Image Patch Similarity*) [17] mierzy percepcyjną odmienną wizualną z wykorzystaniem cech wysokiego poziomu, wyekstrahowanych z głębokiej sieci neuronowej (np. VGG-16). Dzięki wykorzystaniu reprezentacji wyuczonych na dużych zbiorach danych, LPIPS odzwierciedla subiektywne oceny ludzkie w stopniu znacznie wyższym niż klasyczne miary pikselowe [3].

2.7.2 Metryki bezreferencyjne

Metryki bezreferencyjne dokonują oceny jakości lub estetyki obrazu oraz wideo bez konieczności dostępu do materiału wzorcowego.

Naturalny ewaluator jakości obrazu (NIQE, ang. *Natural Image Quality Evaluator*) [18] ocenia stopień naturalności i zniekształceń obrazu poprzez ekstrakcję cech statystycznych (takich jak dane o jasności, kontraście czy krawędziach) i ich dopasowanie do wielowymiarowego rozkładu Gaussa. Metryka ta może jednak wykazywać niestabilność lub generować błędne wyniki w przypadku treści nietypowych, takich jak całkowicie czarne klatki, grafika komputerowa czy specyficzne tekstury o wysokiej częstotliwości.

Wieloskalowy transformer jakości obrazu (MUSIQ, ang. *Multi-scale Image Quality Transformer*) [19] jest bezreferencyjną metryką opartą na architekturze Transformer, dedykowaną do przetwarzania obrazów w ich natywnej rozdzielczości. Dzięki wieloskalowej analizie obrazu podzielonego na fragmenty, MUSIQ eliminuje potrzebę przeskalowywania lub przycinania wejścia, precyzyjnie wychwytyjąc zarówno globalną strukturę, jak i lokalne detale.

CLIPQA [20] wykorzystuje przestrzeń wizyjno-językową wstępnie wytrenowanych modeli CLIP do oceny zarówno aspektów technicznych, jak i właściwości estetycznych obrazu. Metryka ta nie wymaga ręcznie projektowanych cech ani dedykowanego uczenia pod konkretne zadanie, lecz wyznacza jakość poprzez pomiar dopasowania obrazu do przeciwstawnych promptów tekstowych (np. „*Good photo*” vs „*Bad photo*”).

DOVER (ang. *Disentangled Objective Video Quality Evaluator*) [21] stanowi wyspecjalizowaną metrykę oceny jakości sekwencji wideo. Konstrukcja DOVER pozwala na predykcję subiektywnego doświadczenia jakości użytkownika w zróżnicowanych nagraniach wideo poprzez niezależne rozdzielenie analizy na aspekt estetyczny (jakość przestrzenna) oraz techniczny (spójność czasowa i płynność ruchu).

3 Przegląd istniejących rozwiązań

3.1 Metody konwolucyjne i rekurencyjne

Metody konwolucyjne realizują zadanie VSR poprzez podział wideo na nakładające się na siebie segmenty, co umożliwia równoległe przetwarzanie sąsiednich klatek w celu rekonstrukcji klatki docelowej. Reprezentatywny dla tej klasy model *EDVR* [22] wykorzystuje konwolucje deformowalne do adaptacyjnego dopasowania cech przy złożonym ruchu, oraz moduł TSA (*Temporal and Spatial Attention*) do fuzji informacji. Architektury konwolucyjne cechują się dużą stabilnością uczenia i odpornością na dynamiczne zmiany w scenie. Cierpią jednak na ograniczony zasięg kontekstu czasowego oraz wysoki narzut pamięciowy wynikający z wielokrotnego przetwarzania tych samych klatek. Ponadto niezależna rekonstrukcja kolejnych okien sprzyja powstawaniu artefaktów [3], [4].

Metody rekurencyjne traktują wideo jako ciągły strumień danych i przekazują ukryty stan pamięci pomiędzy kolejnymi krokami czasowymi, co pozwala na modelowanie długoterminowych zależności. W modelu *Basic VSR* [23] zastosowano dwukierunkową propagację danych w czasie oraz moduł wyrównywania cech oparty na przepływie optycznym (ang. *optical flow*), który wyrównuje cechy przed przekazaniem ich do bloku rezydualnego. Pozwala to sieci na efektywne uwzględnianie informacji z całej sekwencji. Główną wadą dwukierunkowych metod rekurencyjnych pozostaje jednak wymóg dostępności całej sekwencji wideo przed rozpoczęciem przetwarzania oraz ryzyko stopniowego kumulowania i propagowania błędów rekonstrukcji [3], [4].

3.2 Metody oparte na transformerach

Transformery w VSR wykorzystują mechanizmy samouwagi (ang. *self-attention*) do dynamicznego ważenia i modelowania relacji czasowych wzdłuż całej sekwencji [3]. Przykładowo, model *VSRT* [24] posiada moduł STCSA (*Spatial-Temporal Convolutional Self-Attention*) do ekstrakcji lokalnych cech przestrzennych oraz dwukierunkową warstwę BOFF (*Bidirectional Optical Flow-Based Feed-Forward*), opartą na przepływie optycznym, do wyrównywania cech między klatkami. Alternatywne podejście reprezentuje *VRT* [25], który zamiast przetwarzać klipy sekwencyjnie, rekonstruuje je równoległe. Wykorzystuje on moduł TMSA (*Temporal Mutual Self-Attention*) do jednoczesnej estymacji ruchu, wyrównywania i fuzji cech.

Kluczową zaletą metod opartych na transformerach jest ich zdolność do bezstratnego wychwytywania długoterminowych zależności czasowych oraz możliwość równoległego przetwarzania klatek, co pozwala wyeliminować wąskie gardło rekurencji [25]. Głównym ograniczeniem pozostaje jednak bardzo wysoka złożoność obliczeniowa i ogromne zapotrzebowanie na pamięć w porównaniu z klasycznymi sieciami konwolucyjnymi [3].

3.3 Metody dyfuzyjne

Modele dyfuzyjne, których zasadę działania przedstawiono w podrozdziale 2.2, stanowią klasę rozwiązań generatywnych opartych na procesie iteracyjnego usuwania szumu z reprezentacji w przestrzeni utajonej. Integracja architektury transformerów z modelami dyfuzyjnymi pozwala

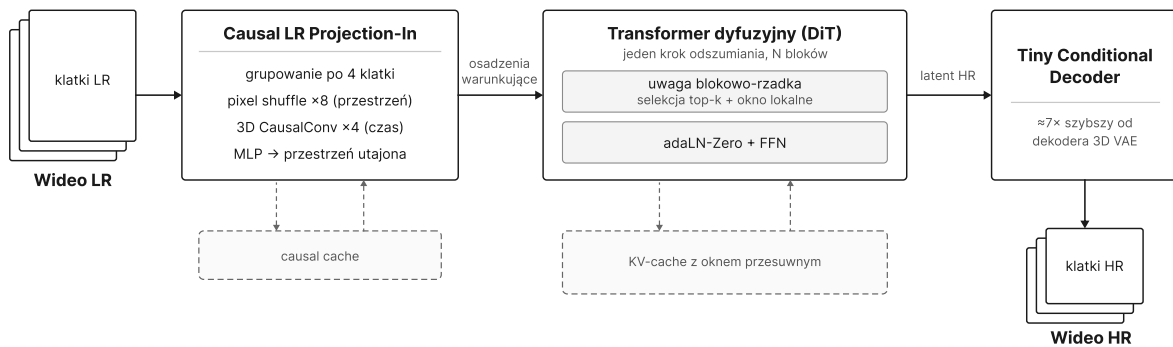
na precyzyjne odtwarzanie złożonych tekstur oraz utrzymanie spójności czasowej między klatkami. Przykładem wielokrokowego podejścia jest model *Upscale-A-Video* [26], który stosuje lokalną i globalną strategię czasową oraz umożliwia sterowanie procesem rekonstrukcji za pomocą promptów tekstowych. Podstawowym ograniczeniem takich metod jest jednak konieczność wykonania kilkudziesięciu sekwencyjnych kroków inferencji, co generuje ogromny narzut obliczeniowy.

W odpowiedzi na ten problem rozwinięto metody jednokrokowe (ang. *one-step*), które skracają cały proces generatywny do pojedynczego przejścia sieci (ang. *forward pass*). Przykładowo, model *DOVE* [27] dostarcza wcześniej wyszkolony, zaawansowany model dyfuzyjny *CogVideoX* poprzez dwuetapową strategię uczenia *latent-pixel*, aby minimalizować różnice między ukrytymi reprezentacjami klatek przewidywanych i HR. Z kolei architektura *SeedVR2* [28] wykorzystuje doszkalanie adversarialne (ang. *Diffusion Adversarial Post-Training*) oraz adaptacyjną samouwagę okienkową (ang. *adaptive window attention*), co pozwala na płynne przetwarzanie sekwencji wideo w wysokich rozdzielczościach bez powstawania artefaktów.

Podstawową zaletą metod jednokrokowych jest więc znaczna redukcja czasu inferencji i zużycia zasobów poprzez zastąpienie wielokrotnych pętli odszumiania pojedynczym przejściem sieci. Odbywa się to jednak kosztem mniej stabilnego treningu oraz większej podatności na artefakty przy złożonym ruchu.

3.4 Model FlashVSR

FlashVSR [1] to architektura przeznaczona do strumieniowego przetwarzania wideo w czasie rzeczywistym, wykorzystująca jednokrokowy model dyfuzyjny. Jego budowę przedstawiono na rysunku 1.



Rysunek 1. Schemat przetwarzania w modelu FlashVSR: projekcja wejściowa, transformer dyfuzyjny z uwagą blokowo-rzadką oraz lekki dekodery warunkowy; opracowanie własne na podstawie [1]

Aby wyeliminować opóźnienia i spadek wydajności, autorzy wprowadzili lekki moduł projekcji wejściowej (*Causal LR Projection-In*). Moduł ten grupuje po cztery klatki LR i poddaje je ośmiokrotnej kompresji przestrzennej za pomocą operacji *pixel shuffle*, a następnie czterokrotnej kompresji czasowej przy użyciu przyczynowych konwolucji 3D (*3D CausalConv*). Dzięki zastosowaniu pamięci podręcznej (ang. *causal cache*), cechy z poprzednich sekwencji

są spójnie przenoszone i rzutowane przez perceptron wielowarstwowy (MLP) bezpośrednio do przestrzeni utajonej transformera dyfuzyjnego.

Głównym rozwiązaniem problemu złożoności obliczeniowej standardowej samouwagi 3D jest zastosowanie blokowo-rzadkiego mechanizmu uwagi z ograniczeniem lokalnym (ang. *Locality-Constrained Sparse Attention*), którego ogólną zasadę omówiono w podrozdziale 2.5.3. FlashVSR wyznacza przybliżoną mapę uwagi poprzez uśrednianie reprezentacji kluczy i wartości, a pełne obliczenia wykonuje wyłącznie dla obszarów o najwyższych wynikach (ang. *top-k*). Dodatkowe nałożenie lokalnych okien przestrzennych ujednolica kodowanie pozycji między treningiem a wnioskowaniem, co umożliwia skalowanie modelu do bardzo wysokich rozdzielczości.

Zamiast dzielić długie wideo na nachodzące na siebie fragmenty FlashVSR przetwarza nagranie w sposób strumieniowy. Płynność i spójność klatek zapewnia pamięć *KV-cache* z mechanizmem okna przesuwającego. Pozwala ona przechowywać najważniejsze cechy obrazu z poprzednich kroków bez konieczności ponownego ich przeliczania.

Ostatnim kluczowym elementem architektury jest zastąpienie ciężkiego, standardowego dekodera 3D VAE autorskim, lekkim dekoderm warunkowym (ang. *Tiny Conditional Decoder*). Zapewnia on około siedmiokrotne przyspieszenie etapu rekonstrukcji obrazu w przestrzeni pikseli w porównaniu z oryginalnym dekoderm sieci Wan, nie powodując przy tym widocznej utraty jakości wizualnej.

3.5 Luka badawcza

Dotychczasowy rozwój metod VSR pokazuje wyraźną ewolucję od szybkich, lecz ograniczonych jakościowo modeli konwolucyjnych i rekurencyjnych, przez dokładniejsze architektury oparte na transformerach, aż po generatywne modele dyfuzyjne. Każda z tych klas rozwiązań charakteryzuje się odmiennym kompromisem pomiędzy jakością generowanego obrazu, wydajnością a zapotrzebowaniem na zasoby sprzętowe.

Metody klasyczne (konwolucyjne i rekurencyjne) oferują najwyższą szybkość i niskie zużycie pamięci, jednak mają tendencję do generowania rozmytych tekstur. Transformery skutecznie modelują długoterminowe zależności czasowe, ale ich złożoność obliczeniowa rośnie kwadratowo wraz z długością sekwencji. Wielokrokowe modele dyfuzyjne zapewniają najwyższy fotorealizm, lecz konieczność wykonywania kilkudziesięciu iteracji pętli odsumowania uniemożliwia ich zastosowanie w trybie interaktywnym.

Architektura *FlashVSR* przełamuje to ograniczenie poprzez sprowadzenie procesu dyfuzji do jednego kroku inferencji oraz wprowadzenie mechanizmów strumieniowych i dedykowanego dekodera. Jakościowe zestawienie cech poszczególnych klas metod przedstawiono w tabeli 1.

Tabela 1. Porównanie klas metod superrozdzielczości wideo pod względem kosztu obliczeniowego, zapotrzebowania na pamięć i jakości rekonstrukcji

Klasa metod (reprezentant)	Czas inferencji	Zużycie pamięci	Jakość rekonstrukcji*	Główne ograniczenie
Konwolucyjne - EDVR [22]	bardzo krótki	umiarkowane, rośnie z $H \cdot W$ i szerokością okna	wysoka wierność, niska jakość percepcyjna - wygładzone tekstury	wąski kontekst czasowy, redundantne przetwarzanie okien
Rekurencyjne - BasicVSR [23]	bardzo krótki	umiarkowane, stan ukryty utrzymywany przez cały klip	wysoka wierność, percepcyjnie średnia	wymaga całego nagrania z góry, akumulacja błędów
Transformerowe - VRT [25], VSRT [24]	średni	wysokie, kwadratowe z długością sekwencji	najwyższa wierność, percepcyjnie średnia	złożoność $O(N^2)$ samouwagi
Dyfuzyjne wielokrokowe - Upscale-A-Video [26]	bardzo długi	umiarkowane, bufor reużywany między krokami	niższa wierność, wysoka jakość percepcyjna	dziesiątki sekwencyjnych przejść sieci
Dyfuzyjne jednokrokowe - FlashVSR [1], DO-VE [27], SeedVR2 [28]	krótki (jedno przejście)	wysokie, wagi + KV-cache + aktywacje jednocześnie	niższa wierność, wysoka jakość percepcyjna	wysoki próg wejścia pamięci, niezależny od długości nagrania

* Wierność mierzona metrykami pełnoreferencyjnymi (PSNR, SSIM), jakość percepcyjna - metrykami LPIPS oraz bezreferencyjnymi.

Oryginalna implementacja modelu *FlashVSR* wykazuje ogromne zapotrzebowanie na pamięć karty graficznej. Wyniki wydajnościowe zaprezentowane przez autorów architektury uzyskano na akceleratorze NVIDIA A100 z 80 GB VRAM. W domyślnej precyzji FP16/BF16 rozmiar wag modelu w połączeniu z buforem KV-cache oraz alokacją pamięci na aktywacje warstw samouwagi sprawia, że próba uruchomienia bazowej wersji na powszechnie dostępnych, konsumenckich kartach graficznych (np. z 10 GB VRAM) kończy się przepełnieniem pamięci.

Głównym problemem jest więc brak możliwości bezpośredniego uruchomienia jednokrokowego modelu *FlashVSR* na sprzęcie o ograniczonych zasobach.

W odpowiedzi na tę lukę zdefiniowano zakres niniejszej pracy, której szczegółowy cel sformułowano w podrozdziale 1.2. Polega on na zaprojektowaniu, implementacji oraz analizie zoptymalizowanego potoku inferencji, który obniży zapotrzebowanie modelu na pamięć do pułapu 10 GB VRAM przy minimalnej utracie jakości obrazu. Kryteria doboru zastosowanych technik oraz ich ostateczne zestawienie omówiono w rozdziale 4.

4 Wybór rozwiązań do implementacji

Analiza przeprowadzona w rozdziale 3 wykazała, że główną barierą w uruchomieniu modelu FlashVSR na sprzęcie konsumenckim jest szczytowe zużycie pamięci podczas pojedynczej inferencji, a nie liczba kroków odszumiania. Przy doborze odpowiednich technik optymalizacyjnych kierowano się dwoma kluczowymi warunkami: pełną kompatybilnością z architekturą Ampere [29] oraz działaniem wyłącznie na etapie inferencji (bez dodatkowego trenowania, kalibracji czy douczania modelu). Na podstawie tych założeń wyodrębniono trzy główne osie optymalizacji: mechanizm uwagi, kwantyzację oraz ograniczenie rozmiaru przetwarzanego fragmentu.

4.1 Mechanizm uwagi

Blokowo-rzadki mechanizm uwagi stosowany w *FlashVSR* składa się z dwóch koncepcyjnie niezależnych elementów: jądra realizującego uwagę gęstą w wywołaniach bez maski blokowej oraz jądra realizującego uwagę rzadką, ograniczoną maską wyznaczoną na podstawie selekcji istotnych par bloków i okien lokalnych. Rozdzielenie ich w warstwie konfiguracji pozwoliło zbadać wpływ każdego z nich. Zasadę działania porównywanych dalej jąder omówiono w podrozdziale 2.5.

Jako wariant odniesienia dla uwagi gęstej przyjęto mechanizm SDPA (*scaled_dot_product_attention*) biblioteki PyTorch [30], stosowany w oryginalnej implementacji. Na architekturze Ampere dobiera on jądro realizujące algorytm *FlashAttention*, w którym redukcja zużycia pamięci wynika wyłącznie z optymalizacji przepływu danych, bez aproksymacji. Wariantem alternatywnym jest *SageAttention* [11], który działa na poziomie pojedynczego wywołania mechanizmu uwagi, a wykorzystywany przez niego format INT8 jest wspierany przez architekturę Ampere.

Wariantem odniesienia dla selekcji rzadkiej jest uwaga blokowo-rzadka autorów *FlashVSR* [1]. Jako alternatywę wybrano *SparseAttention* [13], realizujące to samo zadanie w sposób adaptacyjny. Wykorzystanie architektury *SageAttention* jako wspólnej podstawy sprawia, że oba rozwiązania można łączyć bez żadnych konfliktów implementacyjnych.

Długość pamięci podręcznej kluczy i wartości oraz zasięg okna lokalnego zachowano w wartościach przyjętych przez autorów modelu. Udział wybieranych par bloków jest natomiast zadawany względem stałej rozdzielczości wskazanej przez autorów i przeliczany na powierzchnię faktycznie przetwarzanego fragmentu, przez co efektywna liczba wybieranych bloków zależy od rozmiaru kafla.

4.2 Kwantyzacja

Jako docelowy format kwantyzacji przyjęto INT8. Zastosowanie formatu FP8 wykluczono ze względu na brak wsparcia w docelowej architekturze Ampere, natomiast rozwiązania czterobitowe (takie jak NF4 czy GGUF) odrzucono, gdyż są one optymalizowane głównie pod kątem dużych modeli językowych. Dla INT8 architektura Ampere udostępnia dedykowane jednostki tensorowe, a literatura raportuje dokładność zbliżoną do modelu bazowego przy odpowiednim doborze mechanizmu mapowania [14].

Do realizacji wybrano bibliotekę torchao [31], której działanie polega na konwersji tensorów wag na wyspecjalizowane podtypy, nie wymagając modyfikacji definicji modelu ani eksportu do reprezentacji pośredniej. Kwantyzację można więc włączyć jako opcjonalny krok inicjalizacji potoku. Zbadano dwa jej warianty, odpowiadające podziałowi z podrozdziałów 2.6.2 i 2.6.3: kwantyzację wyłącznie wag oraz kwantyzację wag i aktywacji.

Kwantyzacji poddano wyłącznie transformer dyfuzyjny, który odpowiada za zdecydowaną większość parametrów modelu i jest wywoływany dla każdego przetwarzanego fragmentu.

4.3 Ograniczenie rozmiaru przetwarzanego fragmentu

Zapotrzebowanie na pamięć w obrębie pojedynczego wywołania modelu zależy od liczby tokenów czasoprzestrzennych, a więc od rozdzielczości przetwarzanego fragmentu. Podział klatki na kafle przetwarzane kolejno sprawia, że szczytowe zużycie pamięci przestaje zależeć od rozdzielczości nagrania wejściowego, a zaczyna zależeć wyłącznie od rozmiaru kafła.

Kosztom takiego podejścia jest redundancja obliczeń w obszarze zakładek, ryzyko wystąpienia widocznych artefaktów na granicach niezależnie rekonstruowanych kafli oraz zmiana efektywnej rzadkości uwagi w zależności od ich rozmiaru. Sam sposób łączenia fragmentów obrazu opisano w podrozdziale 6.3.

Analogiczny mechanizm zastosowano w wymiarze czasowym. Mimo strumieniowego trybu pracy modelu bufor wejściowy i wyjściowy długich nagrań pozostają kosztowne pamięciowo, sekwencję dzielono zatem na segmenty o ustalonej długości. Realizację tego podziału opisano w podrozdziale 6.4.

4.4 Zestawienie technik optymalizacji

Wybrane techniki zestawiono w tabeli 2. Ich wykorzystanie można niezależnie konfigurować, co pozwala badać zarówno pojedyncze rozwiązania, jak i ich kombinacje.

Tabela 2. Zestawienie technik wybranych do implementacji

Oś optymalizacji	Wariant odniesienia	Warianty alternatywne	Oczekiwany efekt
Jądro uwagi gęstej	SDPA (FlashAttention) [10]	SageAttention [11]	skrócenie czasu inferencji
Selekcja rzadka	Block Sparse Attention [1]	SpargAttention [13]	skrócenie czasu inferencji
Kwantyzacja	brak (bfloat16)	INT8 wag; INT8 wag i aktywacji	redukcja zużycia pamięci
Kafelkowanie przestrzenne	wylączone	rozmiar kafła i zakładka jako parametry	uniezależnienie szczytu pamięci od rozdzielczości wejścia
Kafelkowanie czasowe	wylączone	długość segmentu i zakładka jako parametry	ograniczenie zużycia pamięci dla długich nagrań

5 Środowisko i projekt systemu

5.1 Platforma sprzętowa

Badania zrealizowano na dwóch stanowiskach sprzętowych o odmiennych rolach. Główną platformą docelową była karta NVIDIA GeForce RTX 3080 (10 GB VRAM, architektura Ampere, compute capability 8.6) zainstalowana w komputerze z procesorem AMD Ryzen 5 5600X oraz 32 GB pamięci RAM - pojemność RAM ma kluczowe znaczenie, gdyż trafiają do niej wyniki pośrednie. Wybrane GPU reprezentuje segment kart konsumenckich, dla których domyślne zapotrzebowanie FlashVSR przekracza dostępny budżet pamięci.

Pomocniczo wykorzystano akcelerator NVIDIA A100, wyposażony w 80 GB pamięci. Uruchomiono na nim konfigurację bazową, czyli wariant bez kafelkowania przestrzennego, którego zapotrzebowanie na pamięć przekracza budżet karty docelowej. Podział zadań między obie platformy opisano w podrozdziale 7.4.

5.2 Środowisko programistyczne

System zaimplementowano w języku Python w wersji 3.11, z wykorzystaniem biblioteki PyTorch 2.10 [30] skompilowanej dla CUDA 12.8. Do zarządzania zależnościami wykorzystano narzędzie `uv`, które zapisuje pełny stan środowiska w pliku blokującym wersje. Zapewnia to odtwarzalność instalacji, istotną przy tak dużej liczbie komponentów wymagających kompilacji ze źródeł.

Trzy biblioteki realizujące warianty mechanizmu uwagi - `block_sparse_attn`, `sageattention` oraz `spas_sage_attn` - instalowane są bezpośrednio z repozytoriów źródłowych i wymagają kompilacji jąder CUDA. Ich instalacja wymagała ustawienia zmiennych środowiskowych określających docelowe architektury GPU oraz wskazania kompilatora w wersji zgodnej z wymaganiami narzędzia `nvcc`. Pozostałe istotne zależności to `torchao` w wersji 0.16 (kwantyzacja), `torchcodec` i `av` (dekodowanie i kodowanie materiału wideo) oraz `pyiqa` (metryki jakości obrazu).

Implementacja referencyjna metryki DOVER wymaga biblioteki PyTorch w wersji sprzed 2.0, niezgodnej z pozostałymi komponentami, dlatego uruchamiano ją w wydzielonym środowisku wirtualnym.

5.3 Architektura pakietu

Implementacja referencyjna modelu *FlashVSR* [32] ma postać zbioru skryptów, w których konfiguracja jest zapisana na stałe w kodzie, co uniemożliwia wygodne porównywanie wariantów. Z tego względu kod zrefaktoryzowano do postaci pakietu o wyraźnie rozdzielonych warstwach odpowiedzialności.

Warstwa `config` zawiera struktury opisujące konfigurację przetwarzania: tryb uwagi, tryb kwantyzacji, parametry kafelkowania przestrzennego i czasowego oraz parametry wejścia i wyjścia. Warstwa `models` obejmuje komponenty modelu: transformer dyfuzyjny, autoenkoder oraz lekki dekodery warunkowy. Warstwa `pipelines` zawiera właściwy potok inferencji, warstwa `processing` odpowiada za jego inicjalizację i orkiestrację przetwarzania nagrania, a warstwa

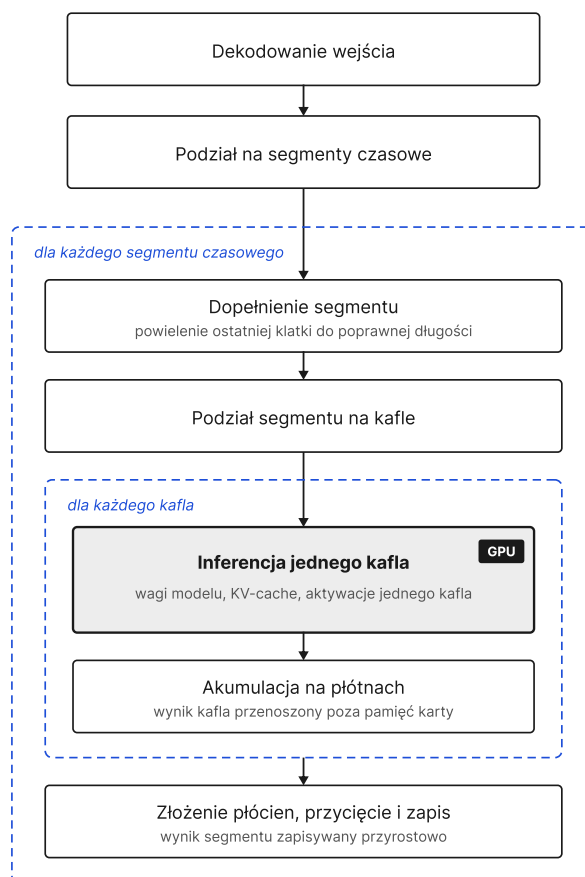
`utils` gromadzi funkcje pomocnicze obliczające podział na kafle, maski mieszania i wymagane wymiary danych.

Poza rdzeniem pakietu wydzielono cztery moduły pomocnicze: `benchmarks` z narzędziami pomiarowymi, `scripts` ze skryptami przetwarzania wsadowego i przygotowania danych, `app` z aplikacją pokazową oraz `tests` z testami jednostkowymi.

Istotnym założeniem projektowym jest oddzielenie budowy potoku od jego wykonania. Inicjalizacja obejmuje wczytanie wag, wybór wariantów uwagi oraz ewentualną kwantyzację, jest więc kosztowna czasowo i pamięciowo. Zbudowany potok może natomiast przetworzyć wiele nagrań bez ponownej inicjalizacji.

5.4 Przepływ przetwarzania

Przebieg przetwarzania nagrania przedstawiono na rysunku 2. Podział na segmenty czasowe i kafle przestrzenne sprawia, że właściwa inferencja wykonywana jest zawsze na pojedynczym kafle, niezależnie od długości i rozdzielczości nagrania wejściowego.



Rysunek 2. Przepływ przetwarzania nagrania w zaimplementowanym potoku

Materiał wejściowy jest najpierw dekodowany, a uzyskana sekwencja klatek dzielona na segmenty czasowe o zadanej długości. Długość segmentu musi spełniać zależność wynikającą z przyczynowej kompresji czasowej autoenkodera, dlatego zbyt krótki segment jest dopełniany powieleniem ostatniej klatki, a nadmiarowe klatki odrzucane po zakończeniu przetwarzania. Mechanizm podziału czasowego omówiono w podrozdziale 6.4.

Każdy segment dzielony jest następnie na kafle przestrzenne, przetwarzane kolejno, niezależnie od siebie. Przed inferencją kafel jest skalowany do rozdzielczości docelowej i dopełniany do wymiarów wymaganych przez podział na łaty, a po jej zakończeniu dopełnienie zostaje usunięte. Gotowe kafle składane są w pełną klatkę przez ważne sumowanie w obszarach nakładania. Podział na kafle oraz sposób ich łączenia opisano w podrozdziale 6.3.

Sama inferencja obejmuje przejście przez transformer dyfuzyjny oraz dekodowanie wyniku do przestrzeni pikseli. To na tym etapie zostały zastosowane wymienne warianty mechanizmu uwagi oraz kwantyzacja, przedstawione odpowiednio w podrozdziale 6.1 i podrozdziale 6.2.

Złożony segment zapisywany jest przyrostowo, a klatki z obszaru zakładki łączone są z segmentem poprzednim. Kluczową decyzją projektową jest przy tym przenoszenie wyniku każdego kafła poza pamięć karty graficznej bezpośrednio po jego przetworzeniu. Na karcie znajdują się jednocześnie wyłącznie wagi modelu, pamięć podręczna kluczy i wartości oraz aktywacje jednego kafła, natomiast płótna akumulacyjne obejmujące jeden segment czasowy utrzymywane są w pamięci operacyjnej.

5.5 Interfejsy systemu

Rdzeń systemu udostępniono przez trzy niezależne punkty wejścia, korzystające z tej samej implementacji potoku.

Pierwszym jest uruchomienie z wiersza poleceń, przeznaczone do przetwarzania pojedynczego nagrania. Drugim jest skrypt przetwarzania wsadowego, stosowany przy generowaniu wyników dla całych zbiorów testowych. Trzecim jest aplikacja pokazowa, złożona z usługi sieciowej zbudowanej w oparciu o bibliotekę FastAPI oraz interfejsu przeglądarkowego wykorzystującego bibliotekę Gradio.

5.6 Jakość kodu

Moduły pomocnicze odpowiedzialne za wyznaczanie podziału na kafle, generowanie masek mieszania oraz obliczanie wymaganych wymiarów danych objęto testami jednostkowymi z wykorzystaniem biblioteki `pytest`. Są to funkcje obliczeniowe, których poprawność decyduje o braku artefaktów w materiale wyjściowym, a jednocześnie dające się testować bez dostępu do karty graficznej. Statyczną analizę kodu i jego formatowanie realizuje narzędzie `ruff`.

6 Implementacja

Oryginalne skrypty inferencyjne przebudowano w jednolity pakiet, co pozwala na łatwe zarządzanie parametrami modelu. Kod rozszerzono również o optymalizacje skracające czas obliczeń i zmniejszające zużycie pamięci. W tym celu zaimplementowano wymienne mechanizmy uwagi, kwantyzację całkowitoliczbową transformera dyfuzyjnego oraz kafelkowanie przestrzenne i czasowe. Kryteria doboru tych trzech osi optymalizacji omówiono w rozdziale 4.

6.1 Wymienne warianty mechanizmu uwagi

W implementacji referencyjnej wybór jądra obliczeniowego nie podlegał konfiguracji. Realizował go łańcuch warunków sterowany obecnością maski blokowej oraz dostępnością pakietów w środowisku. *SageAttention* miało w nim bardzo niski priorytet i było wybierane wyłącznie, gdy *FlashAttention* było niedostępne. Co więcej, pakiet ten nie był wymieniony wśród zadeklarowanych zależności, przez co ścieżka ta pozostawała w praktyce nieosiągalna.

Łańcuch ten zastąpiono jawnym wyborem sterowanym dwoma niezależnymi parametrami, przekazywanymi przy budowie modelu i propagowanymi do wszystkich bloków transformera. Podział ten wynika z trybu pracy mechanizmu uwagi. Może on działać z maską blokową, łączącą najistotniejsze pary bloków z lokalnym oknem przestrzennym, albo w trybie standardowym, bez maskowania. Ścieżkę z maską obsługuje parametr `mask_attn_mode`, wybierający między jądrem `block_sparse_attn` a jądrem `block_sparse_sage2_attn_cuda` z biblioteki *SparseAttention*. Ścieżkę bez maski blokowej obsługuje parametr `attn_mode`, wybierający między funkcją `sageattn` a wywołaniem `scaled_dot_product_attention` biblioteki PyTorch.

Wybrane jądro wymaga obecności odpowiedniej biblioteki w środowisku. Jeżeli biblioteka nie jest dostępna, potok zgłasza ostrzeżenie i wykonuje uwagę gęstą, co zapobiega przerwaniu przetwarzania, lecz zmienia badaną konfigurację.

6.2 Integracja kwantyzacji

Kwantyzację umieszczono w funkcji inicjalizującej potok, po wczytaniu wag i przeniesieniu modelu na kartę graficzną. Realizuje ją pojedyncze wywołanie funkcji `quantize_` biblioteki *torchao*, której przekazuje się *transformer dyfuzyjny* oraz obiekt konfiguracji odpowiadający wybranemu trybowi: `Int8WeightOnlyConfig` dla kwantyzacji wag albo `Int8DynamicActivationInt8WeightConfig` dla kwantyzacji wag i aktywacji.

Biblioteka *torchao* zastępuje tensory wag warstw liniowych własnymi podtypami, przechowującymi wartości skwantyzowane wraz z parametrami mapowania. Podmiana odbywa się w miejscu, bez modyfikacji definicji modelu, dzięki czemu pozostałe komponenty potoku nie wymagały zmian. Kwantyzacji poddano wyłącznie *transformer dyfuzyjny*. Dekoder warunkowy, moduł projekcji wejściowej oraz autoenkoder pozostają w precyzji `bfloat16`.

Kwantyzacja modyfikuje wagi w sposób nieodwracalny, dlatego zmiana jej trybu wymaga ponownej budowy całego potoku. Zasada ta odnosi się również do modyfikacji wariantu uwagi.

6.3 Kafelkowanie przestrzenne

Motywację dla ograniczenia rozmiaru przetwarzanego fragmentu przedstawiono w podrozdziale 4.3. Samo kafelkowanie jest wykonalne dzięki własności odziedziczonej po implementacji referencyjnej: bufor pamięci podręcznej kluczy i wartości inicjalizowane są wartością pustą na początku każdego wywołania potoku, a stan strumieniowy rośnie wyłącznie wzdłuż osi czasu w obrębie jednego wywołania. Każde wywołanie stanowi więc niezależny przebieg. Fragmenty klatki można zatem przetwarzać bez współdzielenia stanu między nimi.

Podział klatki na kafle realizuje funkcja zwracająca listę współrzędnych obszarów wejściowych, wyznaczanych z zadanego rozmiaru kafla i szerokości zakładki. Gdy obszar wykracza poza wymiary klatki, jego brzeg jest przycinany do krawędzi, a brzeg przeciwny cofany tak, aby zachować zadany rozmiar. Zapobiega to przetwarzaniu kafla mniejszych od zadanego rozmiaru, kosztem zwiększonej zakładki przy krawędziach obrazu. Jeżeli natomiast zadany rozmiar przekracza wymiar klatki, cofnięcie nie jest możliwe i kafel zostaje zredukowany do rozmiaru obrazu.

Wyniki poszczególnych kafla łączone są przez akumulację na dwóch płótnach: na pierwszym sumowane są kafle przemnożone przez maskę mieszania, na drugim same wagi maski. Iloraz obu płócien daje średnią ważoną w obszarach nakładania. Maskę nadaje pasom o szerokości zakładki przy każdej krawędzi wagi narastające liniowo ku wnętrzu kafla. Rozwiązanie to nie wymaga rozróżniania kafla brzegowych i wewnętrznych ani znajomości położenia sąsiadów.

6.4 Kafelkowanie czasowe

Implementacja referencyjna obsługiwała długie nagrania odrębną klasą potoku i odrębnym skrypcem. Rozwiązanie to zastąpiono kafelkowaniem czasowym sterowanym konfiguracją, dzięki czemu ten sam potok obsługuje nagrania dowolnej długości.

Sekwencja wejściowa dzielona jest na segmenty o zadanej długości, wyznaczane analogicznie do kafla przestrzennych. Segmenty przetwarzane są kolejno, a wynik zapisywany przyrostowo, co pozwala utrzymywać w pamięci operacyjnej wyłącznie bieżący segment. Jest to konieczne, ponieważ przy długich nagraniach materiał po czterokrotnym powiększeniu przekraczałby pojemność dostępnej pamięci. Każdy segment stanowi niezależny przebieg strumieniowy i nie dziedziczy kontekstu poprzednika, dlatego klatki z obszaru zakładki łączone są przez mieszanie liniowe. Końcówka segmentu poprzedniego i początek segmentu bieżącego sumowane są z wagami zmieniającymi się liniowo wzdłuż osi czasu.

Długość segmentu nie jest dowolna. Czterokrotna przyczynowa kompresja czasowa autoenkodera wymusza wartości spełniające określoną zależność arytmetyczną, wyznaczane przez funkcje pomocnicze pakietu. Zbyt krótki segment jest przed przetworzeniem dopełniany przez powielenie ostatniej klatki, a nadmiarowe klatki są odrzucane po zakończeniu.

6.5 Aplikacja pokazowa

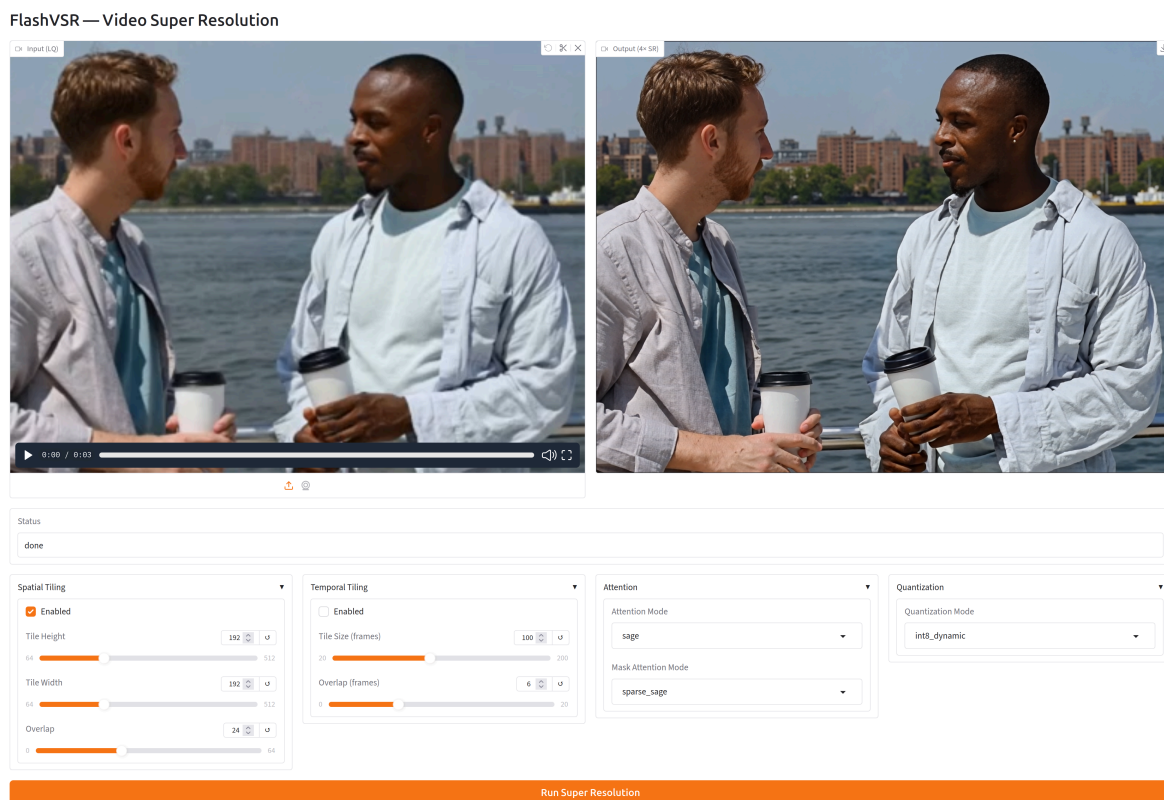
Aplikacja pokazowa umożliwia korzystanie z zaimplementowanych mechanizmów bez znajomości interfejsu programistycznego pakietu oraz porównywanie konfiguracji na dowolnym

nagranie. Składa się z usługi sieciowej oraz interfejsu przeglądarkowego, komunikujących się przez protokół HTTP.

Usługa udostępnia cztery operacje. Zlecenie przetwarzania przyjmowane jest pod adresem `POST /jobs`, wraz z plikiem wejściowym oraz pełną konfiguracją w formacie JSON, obejmującą tryb uwagi, tryb kwantyzacji i parametry kafelkowania, a w odpowiedzi zwracany jest identyfikator zlecenia. Operacja `GET /jobs/{id}` zwraca stan zlecenia wraz z postępem przetwarzania, `GET /jobs/{id}/download` pozwala pobrać gotowe nagranie, a `GET /jobs` wylistować wszystkie zlecenia przyjęte przez usługę. Próba pobrania wyniku zlecenia, którego przetwarzanie jeszcze się nie zakończyło, kończy się odmową ze wskazaniem bieżącego stanu.

Zlecenia obsługiwane są asynchronicznie. Trafiają do kolejki, z której pobiera je pojedynczy proces roboczy uruchamiany przy starcie usługi. Ograniczenie do jednego procesu jest konieczne, ponieważ równoległe przetwarzanie dwóch nagrań wymagałoby jednoczesnego utrzymywania dwóch potoków na karcie, co przekroczyłoby jej budżet pamięci. Właściwa inferencja wykonywana jest w osobnym wątku, dzięki czemu operacje blokujące nie wstrzymują obsługi zapytań o stan zleceń.

Proces roboczy utrzymuje zbudowany potok między kolejnymi zleceniami i odtwarza go wyłącznie wtedy, gdy zmieni się konfiguracja mechanizmu uwagi lub kwantyzacji. Wynika to z własności opisanych w podrozdziałach 6.1 i 6.2: wybór jąder obliczeniowych następuje przy budowie modelu, a kwantyzacja modyfikuje wagi nieodwracalnie. Parametry kafelkowania oraz parametry przetwarzania można natomiast zmieniać między zleceniami bez ponownej inicjalizacji.



Rysunek 3. Interfejs aplikacji pokazowej z widocznymi parametrami trzech osi optymalizacji

Interfejs przeglądarkowy, przedstawiony na rysunku 3, udostępnia wszystkie trzy osie optymalizacji jako kontrolki formularza, pogrupowane w rozwijane sekcje odpowiadające kafelkowaniu przestrzennemu, kafelkowaniu czasowemu, mechanizmowi uwagi oraz kwantyzacji. Suwaki rozmiaru kafla mają skok równy 32, co zapewnia, że przetwarzany fragment nigdy nie wymaga dopełnienia. Po wysłaniu zlecenia interfejs cyklicznie odpytuje usługę o jego stan i wyświetla wynik po zakończeniu przetwarzania.

7 Metodyka badań i testy

7.1 Cel badań i pytania badawcze

Celem przeprowadzonych eksperymentów jest odpowiedź na trzy pytania badawcze, wynikające z założeń przedstawionych w podrozdziale 1.2.

Pierwsze dotyczy wykonalności: przy jakiej konfiguracji potok mieści się w budżecie 10 GB pamięci karty graficznej i jaki jest koszt czasowy tego ograniczenia. Drugie dotyczy udziału poszczególnych osi optymalizacji: jak podmiana jądra uwagi gęstej, podmiana selekcji rzadkiej oraz kwantyzacja wpływają na czas inferencji i szczytowe zużycie pamięci, osobno i w kombinacji. Trzecie dotyczy kosztu jakościowego: o ile pogarsza się rekonstrukcja po włączeniu kolejnych technik i jak koszt ten rozkłada się pomiędzy nimi.

W pracy rozróżnia się dwa punkty odniesienia. Konfiguracja bazowa odpowiada modelowi w postaci udostępnionej przez autorów: przetwarzanie klatki bez podziału na kafle, uwaga gęsta realizowana mechanizmem SDPA, selekcja rzadka zgodna z implementacją autorów, wagi i aktywacje w formacie bfloat16. Służy ona ocenie wykonalności. Konfiguracja odniesienia to ta sama konfiguracja z włączonym kafelkowaniem przestrzennym. Względem niej porównywane są warianty w eksperymentach E1 i E2, które w całości prowadzono na konfiguracjach kafelkowanych.

7.2 Zbiory testowe i przygotowanie danych

Ewaluację jakości przeprowadzono na dwóch zbiorach o rozłącznych rolach.

Zbiór YouHQ40 [27] obejmuje 40 klipów o łącznej długości 1318 klatek, wraz z materiałem referencyjnym, na którym możliwe było wyznaczenie metryk pełnreferencyjnych. Odpowiadający mu materiał o niskiej rozdzielczości wygenerowano skryptem `generate_lq.py`, odtwarzającym potok degradacji przyjęty w pracy RealBasicVSR [5].

Zbiór VideoLQ [5] obejmuje 50 klipów po 100 klatek zawierających degradacje rzeczywiste, bez materiału referencyjnego. Wyznaczano na nim wyłącznie metryki bezreferencyjne. Jego rolą jest sprawdzenie, czy wnioski wyciągnięte z materiału degradowanego syntetycznie przenoszą się na nagrania rzeczywiste.

7.3 Metryki

7.3.1 Jakość rekonstrukcji

Zastosowano zestaw siedmiu metryk opisanych w podrozdziale 2.7. Metryki PSNR, SSIM, LPIPS, NIQE, MUSIQ oraz CLIPQA wyznaczano biblioteką `pyiqa`, osobno dla każdej klatki wyjściowej. Wartości uśredniano najpierw w obrębie klipu, a następnie po klipach zbioru. Przy klipach o różnej długości procedura ta daje inny wynik niż uśrednienie po wszystkich klatkach i nadaje każdemu klipowi jednakową wagę niezależnie od jego długości.

Metrykę DOVER wyznaczano dla całych klipów, a nie dla pojedynczych klatek, ponieważ ocenia ona również spójność czasową. Ze względu na niezgodność wersji biblioteki PyTorch, opisaną w podrozdziale 5.2, uruchamiano ją w wydzielonym środowisku wirtualnym, na

materiale zapisanym wcześniej przez główny potok, a wyniki obu środowisk łączono skryptem agregującym.

Metryki pełnreferencyjne wyznaczono wyłącznie dla zbioru YouHQ40. W zestawieniach dotyczących zbioru VideoLQ odpowiadające im pola pozostają puste.

7.3.2 Wydajność

Rejestrowano pięć wielkości: czas budowy potoku, czas inferencji całej sekwencji, czas przypadający na klatkę, ilość pamięci zajętej po zakończeniu inicjalizacji oraz szczytowe zużycie pamięci karty graficznej.

Jako miarę zużycia pamięci przyjęto wartość `max_memory_reserved`, czyli szczytowy rozmiar puli zarezerwowanej przez alokator, a nie sumę rozmiarów żywych tensorów raportowaną przez `max_memory_allocated`. O wystąpieniu przepełnienia decyduje rozmiar puli, ponieważ to on odzwierciedla pamięć faktycznie odebraną sterownikowi, wraz z fragmentacją. Wartości zużycia pamięci podawane są w mebibajtach (MiB), zgodnie z jednostką raportowaną przez bibliotekę PyTorch.

7.4 Procedura pomiaru wydajności

Pomiary wykonywano na materiale o długości 101 klatek. Liczba ta odpowiada scenariuszowi raportowanemu przez autorów modelu FlashVSR. Klatki wycinano centralnie z klipu testowego do zadanej rozdzielczości.

Materiał wczytywano do pamięci operacyjnej przed rozpoczęciem pomiaru, a wynik odrzucano bez zapisu. Mierzony czas nie obejmuje zatem dekodowania wejścia ani kodowania wyjścia, co ogranicza wpływ operacji wejścia-wyjścia na porównanie konfiguracji.

Potok budowano jednorazowo dla każdej konfiguracji, a czas budowy mierzono osobno i nie wliczano go do czasu inferencji. Jest to uzasadnione trybem pracy systemu opisanym w podrozdziale 5.3 - zbudowany potok przetwarza wiele nagrań bez ponownej inicjalizacji, więc jej koszt rozkłada się na całą sesję.

Przed pomiarem wykonywano przebieg rozgrzewkowy, pochłaniający kompilację jąder obliczeniowych i pierwszą alokację buforów pamięci podręcznej. Licznik szczytowego zużycia pamięci zerowano przed każdym przebiegiem mierzonym, a pulę alokatora zwalniano między konfiguracjami. Raportowaną wartością czasu jest mediana przebiegów mierzonych, a wartością zużycia pamięci ich maksimum. Wybór maksimum wynika z faktu, że o mieszczeniu się w budżecie decyduje przypadek najgorszy, a nie przeciętny.

Czas inferencji mierzono wyłącznie na karcie RTX 3080, ponieważ zależy on od architektury karty i nie przenosi się między platformami. Na tej samej karcie mierzono szczytowe zużycie pamięci we wszystkich konfiguracjach, które się na niej uruchamiają. Wyjątkiem jest konfiguracja bazowa. Próba jej uruchomienia na karcie docelowej kończy się przepełnieniem pamięci, dlatego zapotrzebowanie zmierzono dodatkowo na akceleratorze A100. Na tym samym akceleratorze generowano materiał wyjściowy dla zbiorów testowych, na którym wyznaczano metryki jakości, ze względu na czas potrzebny na przetworzenie obu zbiorów.

7.5 Plan eksperymentów

Przeprowadzono trzy eksperymenty, zestawione w tabeli 3. W każdym z nich jawnie ustalono wielkość zmienianą oraz parametry utrzymywane na stałym poziomie.

Tabela 3. Zestawienie przeprowadzonych eksperymentów

Eksperyment	Zmieniane	Stałe	Mierzone
E1	dwanaście kombinacji: jądro uwagi gęstej (2) $\times$ selekcja rzadka (2) $\times$ tryb kwantyzacji (3)	kafel 192×192 , zakładka 24, wejście 192×352 , 101 klatek	czas, pamięć
E2	rozmiar kafla: 128, 160, 192, 224, 256	zakładka 24, konfiguracja odniesienia, wejście 256×448 , 101 klatek	czas, pamięć, granica wykonalności
E3	pięć konfiguracji przyrostowych oraz dwa rozmiary kafla	oba zbiory testowe, kafel 192 poza wariantami rozmiaru	siedem metryk jakości

W eksperymencie E1 przyjęto rozdzielczość wejściową 192×352 , odpowiadającą scenariuszowi, dla którego autorzy modelu FlashVSR raportują szczytowe zużycie pamięci. Pozwala to zestawić uzyskane wartości bezpośrednio z liczbą przytoczoną w podrozdziale 1.1. Porównywalność samych konfiguracji zapewnia stały rozmiar kafla, od którego zależy szczytowe zużycie pamięci.

Eksperyment E2 rozszerzono o pięć przebiegów uzupełniających. Konfigurację bazową uruchomiono przy dwóch rozdzielczościach wejściowych: 192×352 , odpowiadającej scenariuszowi autorów modelu, oraz 256×448 , przyjętej w pozostałych pomiarach tego eksperymentu. Każdą z nich uruchomiono na obu platformach - na karcie docelowej w celu sprawdzenia, czy mieści się w jej budżecie pamięci, oraz na akceleratorze A100, w celu określenia skali zapotrzebowania. Piąty przebieg wykonano dla kafla 224×224 , przy którym konfiguracja odniesienia kończy się przepełnieniem pamięci, lecz z włączoną kwantyzacją i podmienionymi jądrami uwagi. Celem było ustalenie, czy badane techniki przesuwają granicę wykonalności, a nie jedynie obniżają zużycie pamięci w konfiguracjach już wykonalnych.

Konfiguracje eksperymentu E3 dobrano tak, by tworzyły łańcuch przyrostowy, w którym każda kolejna konfiguracja różni się od poprzedniej jedną zmianą. Punktem wyjścia jest konfiguracja bazowa, do której dodawane jest kolejno kafelkowanie przestrzenne, podmiana jądra uwagi gęstej, podmiana selekcji rzadkiej oraz kwantyzacja wag i aktywacji. Konstrukcja ta pozwala przypisać zmianę jakości konkretnej technice, zamiast porównywać konfiguracje różniące się kilkoma parametrami naraz.

8 Wyniki i ich omówienie

8.1 Wpływ mechanizmu uwagi i kwantyzacji

Wyniki eksperymentu E1 zestawiono w tabeli 4, a zmiany względem konfiguracji odniesienia w tabeli 5. Wszystkie konfiguracje zmierzono przy kaflu 192×192 i rozdzielczości wejściowej 192×352 , dla 101 klatek.

Tabela 4. Czas inferencji i szczytowe zużycie pamięci dla dwunastu kombinacji mechanizmu uwagi i trybu kwantyzacji (wejście 192×352 , kafel 192, 101 klatek)

Jądro gęste	Selekcja rzadka	Kwantyzacja	Czas [s]	s/klatkę	Szczyt [MiB]
SDPA	Block Sparse Attention	brak	26,89	0,266	9 110
SDPA	Block Sparse Attention	INT8 wag	27,87	0,276	8 780
SDPA	Block Sparse Attention	INT8 wag i akt.	34,84	0,345	8 212
SageAttention	Block Sparse Attention	brak	27,34	0,271	9 292
SageAttention	Block Sparse Attention	INT8 wag	28,15	0,279	8 764
SageAttention	Block Sparse Attention	INT8 wag i akt.	35,15	0,348	8 240
SDPA	SpargAttention	brak	23,43	0,232	9 286
SDPA	SpargAttention	INT8 wag	24,22	0,240	8 766
SDPA	SpargAttention	INT8 wag i akt.	31,12	0,308	8 112
SageAttention	SpargAttention	brak	23,59	0,234	9 294
SageAttention	SpargAttention	INT8 wag	24,35	0,241	8 766
SageAttention	SpargAttention	INT8 wag i akt.	31,29	0,310	8 112

Tabela 5. Zmiany względem konfiguracji odniesienia

Konfiguracja	Czas (wzgl.)	Pamięć	Pamięć (wzgl.)
SDPA / Block Sparse Attention / brak	0,0%	0 MiB	0,0%
SDPA / Block Sparse Attention / INT8 wag	+3,6%	−330 MiB	−3,6%
SDPA / Block Sparse Attention / INT8 wag i akt.	+29,6%	−898 MiB	−9,9%
SageAttention / Block Sparse Attention / brak	+1,7%	+182 MiB	+2,0%
SageAttention / Block Sparse Attention / INT8 wag	+4,7%	−346 MiB	−3,8%
SageAttention / Block Sparse Attention / INT8 wag i akt.	+30,7%	−870 MiB	−9,5%
SDPA / SpargAttention / brak	−12,9%	+176 MiB	+1,9%
SDPA / SpargAttention / INT8 wag	−9,9%	−344 MiB	−3,8%
SDPA / SpargAttention / INT8 wag i akt.	+15,7%	−998 MiB	−11,0%
SageAttention / SpargAttention / brak	−12,3%	+184 MiB	+2,0%
SageAttention / SpargAttention / INT8 wag	−9,4%	−344 MiB	−3,8%
SageAttention / SpargAttention / INT8 wag i akt.	+16,4%	−998 MiB	−11,0%

Zestawienie pozwala rozdzielić wpływ trzech osi optymalizacji, ponieważ każda z nich zmienia inną wielkość.

Podmiana jądra uwagi gęstej nie przynosi korzyści. Zastąpienie mechanizmu SDPA biblioteką *SageAttention* wydłuża czas o 1,7% przy *Block Sparse Attention* i o 0,7% przy

SparseAttention, a szczytowe zużycie pamięci podnosi o około 180 MiB. Wynik ten jest niezgodny z oczekiwaniem sformułowanym w tabeli 2. Możliwym wyjaśnieniem jest niewielki udział wywołań bez maski blokowej w całkowitym koszcie obliczeń. Jeżeli w badanym modelu przeważają wywołania z maską, obsługiwane przez ścieżkę rzadką, to przyspieszenie ścieżki gęstej dotyczy niewielkiej części pracy, a narzut kwantyzacji macierzy Q i K , wykonywanej przy każdym wywołaniu, nie zostaje zamortyzowany.

Podmiana selekcji rzadkiej skraca czas. Zastąpienie uwagi blokowo-rzadkiej autorów modelu biblioteką *SparseAttention* skraca czas inferencji o 12,9% przy niezmiennym trybie kwantyzacji, kosztem wzrostu szczytu pamięci o około 176 MiB. Jest to jedyne z badanych rozwiązań, które faktycznie przyspiesza przetwarzanie. Skrócenie czasu wynika prawdopodobnie z pomijania części obliczeń przez filtr opisany w podrozdziale 2.5.4. Pomiar nie pozwala jednak stwierdzić, jaki jest udział samej selekcji, a jaki kwantyzacji macierzy Q i K , którą *SparseAttention* dziedziczy po *SageAttention*. Nie wiadomo też, czy metoda ta pomija więcej obliczeń niż metoda autorów modelu. Za tą drugą możliwością przemawia obserwacja z podrozdziału 8.4. Podmiana selekcji rzadkiej jest tam jedyną zmianą o zauważalnym wpływie na jakość.

Kwantyzacja obniża zużycie pamięci i wydłuża czas. Kwantyzacja samych wag zmniejsza szczyt o 330 MiB i wydłuża czas o 3,6%. Objęcie kwantyzacją również aktywacji zmniejsza szczyt o 898 MiB, czyli blisko trzykrotnie więcej, ale wydłuża czas o 29,6%. Wynika to z dodatkowych operacji wykonywanych w tym wariancie. Parametry mapowania aktywacji wyznaczane są przy każdym wywołaniu, a wokół każdego mnożenia macierzowego dochodzą kwantyzacja i dekwantyzacja.

Efekty tych osi łączą się. Najniższy szczyt w całym zestawieniu, 8112 MiB, osiąga konfiguracja łącząca *SparseAttention* z kwantyzacją wag i aktywacji. Jest to o 11,0% mniej niż w konfiguracji odniesienia, przy czasie dłuższym o 15,7%. Na uwagę zasługuje jednak inna konfiguracja. *SparseAttention* z kwantyzacją samych wag daje szczyt 8766 MiB i czas 24,22 s. Jest więc jednocześnie oszczędniejsza pamięciowo i o 9,9% szybsza od konfiguracji wyjściowej. To jedyny badany wariant poprawiający obie mierzone wielkości naraz.

8.2 Wpływ rozmiaru kafła

Wyniki eksperymentu E2 zestawiono w tabeli 6. Pomiary wykonano przy rozdzielczości wejściowej 256×448 i zakładce 24 pikseli. Kolumna redundancji podaje stosunek łącznej powierzchni przetworzonych kafli do powierzchni klatki.

Tabela 6. Czas inferencji i szczytowe zużycie pamięci w funkcji rozmiaru kafła (wejście 256×448 , zakładka 24)

Rozmiar kafła	Liczba kafli	Redundancja	Czas [s]	s/klatkę	Czas/kafel [s]	Szczyt [MiB]
128	15	2,14	81,32	0,805	5,42	6 472
160	8	1,79	72,78	0,721	9,10	7 386
192	6	1,93	81,62	0,808	13,60	9 292
224	6	2,62	przepelnienie	—	—	—
256	2	1,14	przepelnienie	—	—	—

Szczytowe zużycie pamięci rośnie monotonicznie wraz z rozmiarem kafla, zgodnie z założeniem przyjętym w podrozdziale 4.3. Czas inferencji zachowuje się natomiast niemonotonicznie. Kafel 160 daje wynik 72,8 s, natomiast kafle 128 i 192 dają odpowiednio 81,3 s i 81,6 s.

Na czas wpływają dwa czynniki o zbliżonej wadze. Pierwszym jest redundancja obliczeń w obszarach zakładek, wynosząca 2,14 dla kafla 128, 1,79 dla kafla 160 i 1,93 dla kafla 192. Drugim jest koszt jednostkowy, rosnący wraz z powierzchnią kafla od $3,31 \cdot 10^{-4}$ s na piksel dla kafla 128 do $3,69 \cdot 10^{-4}$ s dla kafla 192. Czynniki te przy kaflach 128 i 192 niemal dokładnie się znoszą: pierwszy przetwarza 1,11 raza większą powierzchnię, lecz kosztuje o 10% mniej na piksel.

Redundancja nie zmienia się monotonicznie wraz z rozmiarem kafla, lecz zależy od tego, jak siatka kafla dzieli klatkę. Gdy kafel nie mieści się w klatce, jego początek przesuwany jest wstecz, zgodnie z mechanizmem opisanym w podrozdziale 6.3, co przy rozmiarach źle dopasowanych do jej wymiarów prowadzi do bardzo szerokich zakładek. Widać to na przykładzie kafla 224, który daje tę samą liczbę kafla co kafel 192, przy większej powierzchni każdego z nich, przez co jego redundancja sięga 2,62.

Dobór rozmiaru kafla wymaga zatem uwzględnienia obu czynników, przy czym redundancję można wyznaczyć z samej geometrii podziału, bez uruchamiania modelu. Uzyskane wartości odnoszą się przy tym wyłącznie do rozdzielczości 256×448 i przy innej rozdzielczości korzystniejszy może być inny rozmiar.

8.3 Granica wykonalności

Konfiguracja bazowa kończy się na karcie docelowej przepełnieniem pamięci przy rozdzielczości 192×352 oraz 256×448 . W obu przypadkach awaria następuje w pierwszym kroku odsumowania, w funkcji aktywacji wewnątrz bloku transformera. Modelu w postaci udostępnionej przez autorów nie da się zatem uruchomić na karcie konsumenckiej nawet w scenariuszu, dla którego autorzy podają wyniki wydajnościowe.

Skalę zapotrzebowania określono na akceleratorze A100, gdzie obie konfiguracje wykonują się poprawnie. Wyniki zestawiono w tabeli 7.

Tabela 7. Zapotrzebowanie konfiguracji bazowej, zmierzone na akceleratorze A100

Wejście	Szczyt [MiB]	Czas [s]	Krotność pojemności RTX 3080
192×352	14 844	9,21	1,50×
256×448	23 512	13,92	2,38×

Pomiary na akceleratorze A100 pozwalają określić skalę deficytu. Przy rozdzielczości wskazanej przez autorów zapotrzebowanie wynosi 14 844 MiB, przy rozdzielczości bliższej typowym nagraniom rośnie do 23 512 MiB. Wartości te przekraczają pojemność karty docelowej odpowiednio około półtorakrotnie i blisko dwuipółkrotnie. Deficyt jest zatem na tyle duży, że nie dałoby się go pokryć samą kwantyzacją ani optymalizacją mechanizmu uwagi, których łączny efekt zmierzony w eksperymencie E1 nie przekracza 11%.

Zmierzona wartość 14 844 MiB jest wyższa od 11,13 GB raportowanych przez autorów dla tej samej rozdzielczości. Rozbieżność wynika najpewniej z przyjętej miary zużycia pamięci, omówionej w podrozdziale 7.3.2. Przyjęta tu miara jest ostrożniejsza, ponieważ o wystąpieniu przepełnienia decyduje rozmiar puli zarezerwowanej.

Kafelkowanie przestrzenne uniezależnia szczytowe zużycie pamięci od rozdzielczości nagrania. Potwierdzają to wyniki eksperymentów E1 i E2. Przy kaflu 192×192 szczyt wynosi 9110 MiB dla wejścia 192×352 i 9292 MiB dla wejścia 256×448 , mimo że powierzchnia klatki rośnie o 70%. Różnica mieści się w rozrzucie obserwowanym między powtórzeniami tej samej konfiguracji. Dla porównania, ta sama zmiana rozdzielczości podnosi szczyt konfiguracji bazowej o 8668 MiB (tabela 7).

Same techniki obliczeniowe przesuwają natomiast granicę wykonalności o jeden krok siatki rozmiarów kafla, co przedstawiono w tabeli 8.

Tabela 8. Przesunięcie granicy wykonalności: rozmiar kafla niewykonalny w konfiguracji odniesienia staje się wykonalny po zastosowaniu badanych technik (wejście 256×448 , 101 klatek)

Konfiguracja	Kafel	Wynik	Czas [s]	Szczyt [MiB]
odniesienia	192	wykonalna	81,62	9 292
odniesienia	224	przepełnienie	—	—
SageAttention / SpargeAttention / INT8 wag i akt.	224	wykonalna	132,80	8 920

W konfiguracji odniesienia kafel 224 kończy się przepełnieniem pamięci. Ten sam kafel po zastosowaniu *SparseAttention* oraz kwantyzacji wag i aktywacji przetwarzany jest ze szczytem 8920 MiB, a więc z wyraźnym zapasem względem konfiguracji, przy której nastąpiło przepełnienie.

8.4 Jakość rekonstrukcji

Wyniki eksperymentu E3 dla zbioru YouHQ40 zestawiono w tabeli 9. Kolejne wiersze odpowiadają konfiguracjom łańcucha przyrostowego opisanego w podrozdziale 7.5, w którym każda różni się od poprzedniej jedną zmianą. Zmiany metryk między kolejnymi konfiguracjami przedstawiono w tabeli 10.

Tabela 9. Metryki jakości dla zbioru YouHQ40, wartości uśrednione po klipach

Konfiguracja	PSNR↑	SSIM↑	LPIPS↓	NIQE↓	MUSIQ↑	CLIPQA↑	DOVER↑
bazowa	21,79	0,591	0,393	4,245	60,78	0,486	12,52
+ kafelkowanie 192	21,54	0,599	0,410	4,149	59,42	0,443	11,78
+ SageAttention	21,54	0,599	0,410	4,149	59,41	0,444	11,66
+ SpargeAttention	21,55	0,599	0,411	4,167	58,94	0,442	11,63
+ INT8 wag i aktywacji	21,54	0,599	0,410	4,157	58,97	0,440	11,60

Tabela 10. Zmiana metryk względem konfiguracji poprzedniej, zbiór YouHQ40

Konfiguracja	PSNR↑	SSIM↑	LPIPS↓	NIQE↓	MUSIQ↑	CLIPQA↑	DOVER↑
+ kafelkowanie 192	−0,25	+0,008	+0,016	−0,096	−1,36	−0,043	−0,73
+ SageAttention	0,00	0,000	0,000	0,000	−0,01	0,000	−0,12
+ SpargeAttention	+0,01	0,000	+0,001	+0,018	−0,47	−0,001	−0,03
+ INT8 wag i aktywacji	−0,01	0,000	0,000	−0,009	+0,03	−0,002	−0,03

Utrata jakości przypada niemal wyłącznie na wprowadzenie kafelkowania przestrzennego. Trzy kolejne podmiany, obejmujące jądro uwagi gęstej, selekcję rzadką oraz kwantyzację wag i aktywacji, zmieniają metryki nieznacznie. Wyjątkiem jest spadek MUSIQ o 0,47 przy *SpargeAttention*, wyraźny na tle pozostałych podmian, choć blisko trzykrotnie mniejszy od spadku spowodowanego kafelkowaniem.

Badane optymalizacje obliczeniowe są zatem w przybliżeniu neutralne jakościowo, a kompromis między jakością a zużyciem pamięci sprowadza się do decyzji o podziale klatki na kafle. W szczególności kwantyzacja wag i aktywacji, redukująca szczyt zużycia pamięci o blisko 1 GiB, nie pogarsza rekonstrukcji w stopniu mierzalnym przyjętym zestawem metryk.

Wpływ rozmiaru kafla przedstawiono w tabeli 11. Pozostałe parametry odpowiadają w tych konfiguracjach drugiemu wierszowi tabeli 9, zmieniany jest wyłącznie rozmiar kafla.

Tabela 11. Metryki jakości dla różnych rozmiarów kafla, zbiór YouHQ40.

Konfiguracja	PSNR↑	SSIM↑	LPIPS↓	NIQE↓	MUSIQ↑	CLIPQA↑	DOVER↑
kafelkowanie 192	21,54	0,599	0,410	4,149	59,42	0,443	11,78
kafelkowanie 160	21,47	0,594	0,415	4,092	60,46	0,468	12,18
kafelkowanie 128	21,49	0,592	0,427	4,122	59,78	0,474	11,17

Porównanie nie daje jednoznacznego uporządkowania. Kafel 192 wypada najlepiej we wszystkich trzech metrykach pełnreferencyjnych, natomiast kafle mniejsze osiągają lepsze wyniki metryk bezreferencyjnych: NIQE i MUSIQ są najkorzystniejsze dla kafla 160, a CLIPQA dla kafla 128.

Wyniki dla zbioru VideoLQ, zawierającego nagrania o degradacjach rzeczywistych, zestawiono w tabeli 12, a zmiany między kolejnymi konfiguracjami w tabeli 13.

Tabela 12. Metryki bezreferencyjne dla zbioru VideoLQ, wartości uśrednione po klipach

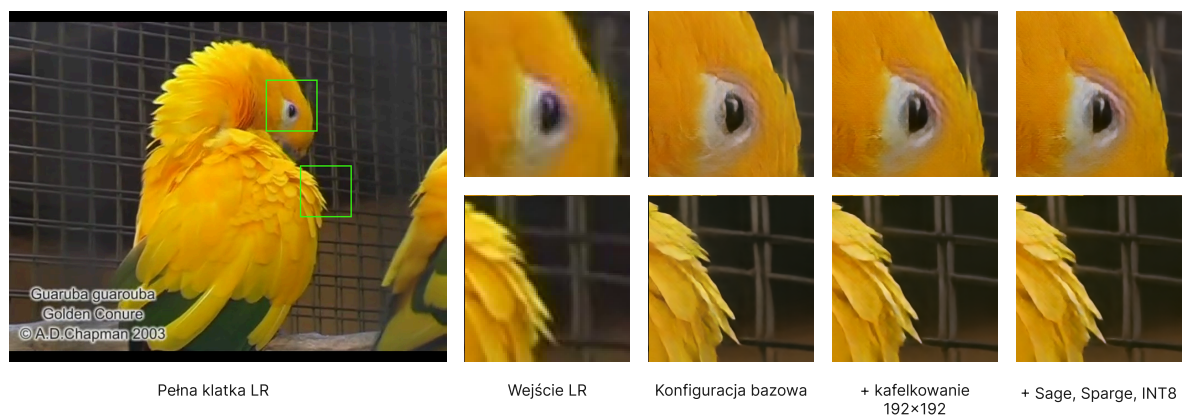
Konfiguracja	NIQE↓	MUSIQ↑	CLIPQA↑	DOVER↑
bazowa	3,937	52,02	0,402	8,02
+ kafelkowanie 192	3,870	50,93	0,391	7,86
+ SageAttention	3,869	50,94	0,391	7,91
+ SpargeAttention	3,865	50,91	0,393	7,87
+ INT8 wag i aktywacji	3,881	50,90	0,392	7,84

Tabela 13. Zmiana metryk względem konfiguracji poprzedniej, zbiór VideoLQ

Konfiguracja	NIQE↓	MUSIQ↑	CLIPQA↑	DOVER↑
+ kafelkowanie 192	-0,067	-1,09	-0,011	-0,16
+ SageAttention	0,000	+0,01	0,000	+0,05
+ SpargeAttention	-0,004	-0,04	+0,002	-0,04
+ INT8 wag i aktywacji	+0,015	-0,01	-0,001	-0,02

Wyniki eksperymentu przeprowadzonego na zbiorze VideoLQ Potwierdzają zaobserwowaną zależność. Na wprowadzenie kafelkowania przypada zdecydowana większość zmiany każdej z metryk. Wniosek o neutralności jakościowej optymalizacji obliczeniowych przenosi się zatem na nagrania rzeczywiste.

Porównanie wizualne dla dwóch klipów o odmiennym charakterze treści przedstawiono na rysunkach 4 i 5. Kolumny przedstawiają materiał wejściowy oraz trzy wybrane konfiguracje z tabeli 12: bazową, bazową z kafelkowaniem oraz konfigurację ze wszystkimi optymalizacjami. W obu przykładach wyniki pozostają nierozróżnialne, co potwierdza wniosek o neutralności jakościowej optymalizacji obliczeniowych.

**Rysunek 4.** Porównanie jakościowe dla zbioru VideoLQ, klip 6, klatka 13.**Rysunek 5.** Porównanie jakościowe dla zbioru VideoLQ, klip 8, klatka 0.

8.5 Podsumowanie wyników

Trzy badane osie optymalizacji okazały się pełnić odmienne role. Kafelkowanie przestrzenne jako jedyne uniezależnia szczytowe zużycie pamięci od rozdzielczości nagrania oraz ponosi cały mierzalny koszt jakościowy. Selekcja rzadka oddziałuje przede wszystkim na czas inferencji, skracając go o blisko 13% przy niemal niezmiennych metrykach. Kwantyzacja oddziałuje na pamięć, obniżając szczyt o 898 MiB w wariancie obejmującym aktywację. Kafelkowanie decyduje więc o wykonalności, a pozostałe dwie osie o kompromisie między czasem a zapasem pamięci w konfiguracji już wykonalnej.

9 Podsumowanie i wnioski

9.1 Podsumowanie prac

Implementacja referencyjna modelu FlashVSR została zrefaktoryzowana ze zbioru skryptów z utrwalonymi ustawieniami do postaci modularnego pakietu z konfigurowalnym potokiem inferencji. W pakiecie zaimplementowano trzy osie optymalizacji, z których każda może być przełączana niezależnie: wymienne warianty mechanizmu uwagi, całkowitoliczbowa kwantyzacja transformera dyfuzyjnego oraz kafelkowanie przestrzenne i czasowe. Opracowano metodykę pomiarową obejmującą czas inferencji, szczytowe zużycie pamięci oraz zestaw siedmiu metryk jakości rekonstrukcji. Przeprowadzono trzy eksperymenty, przy czym ocenę jakości rekonstrukcji wykonano na dwóch zbiorach testowych: jednym z degradacjami syntetycznymi, a drugim z rzeczywistymi.

Cel pracy, którym było umożliwienie inferencji jednokrokowego modelu superrozdzielczości wideo opartego na dyfuzji na karcie graficznej wyposażonej w 10 GB pamięci VRAM, został zrealizowany. Modelu w postaci udostępnionej przez autorów nie da się uruchomić na karcie docelowej przy żadnej z badanych rozdzielczości wejściowych. W konfiguracji zalecanej w podrozdziale 9.2 model przetwarza ten sam materiał w dostępnym budżecie pamięci, kosztem wydłużenia czasu inferencji i akceptowalnej utraty jakości rekonstrukcji.

9.2 Wnioski badawcze

Warunkiem koniecznym wykonalności jest kafelkowanie przestrzenne. Pomiary na akceleratorze A100 pokazały, że zapotrzebowanie na pamięć konfiguracji bazowej przekraczało pojemność karty docelowej od 1,5 do blisko 2,4 raza. Zastosowanie samych technik obliczeniowych zmniejszyło zużycie pamięci o nie więcej niż 11%. Wartość tę zmierzono w konfiguracji kafelkowanej, więc nie przenosi się ona wprost na przetwarzanie pełnej klatki, gdzie udział aktywacji w szczycie jest wyższy. Nawet przy założeniu dwukrotnie większej redukcji deficyt pozostałby niepokryty. Kafelkowanie przestrzenne jest jedyną techniką, która uniezależnia szczytowe zużycie pamięci od rozdzielczości wejściowej, dlatego jest wymagane do uruchomienia modelu. Techniki obliczeniowe pozwalają jednak zyskać zapas pamięci i przesunąć granicę wykonalności o jeden krok siatki w rozmiarach kafla.

Wpływ poszczególnych osi optymalizacji różni się od pierwotnych założeń. Zastąpienie gęstego jądra uwagi nie przynosi żadnych korzyści, wbrew oczekiwaniom przedstawionym w tabeli 2. Z kolei podmiana mechanizmu selekcji rzadkiej skraca czas inferencji o 12,9% i jest jedyną badaną techniką, która faktycznie przyspiesza przetwarzanie. Kwantyzacja oddziałuje na zużycie pamięci, przy czym oba jej warianty zachowują się odmiennie: kwantyzacja samych wag obniża szczyt o 330 MiB przy wydłużeniu czasu o 3,6%, natomiast objęcie nią również aktywacji obniża szczyt o 898 MiB, lecz wydłuża czas o 29,6%.

Spadek jakości rekonstrukcji wynika niemal wyłącznie z zastosowania kafelkowania. Analizując łańcuch przyrostowych konfiguracji, to właśnie wprowadzenie kafelkowania odpowiada za całą zmianę metryki PSNR oraz za większość spadków w pozostałych wskaźnikach. Trzy kolejne modyfikacje obliczeniowe zmieniają wyniki w sposób marginalny.

Oznacza to, że kompromis między jakością obrazu a zużyciem pamięci sprowadza się w praktyce do decyzji o kafelkowaniu, a nie do wyboru poszczególnych technik obliczeniowych.

Ustalenia te wyznaczają kolejność decyzji konfiguracyjnych. Jako pierwszy należy ustalić rozmiar kafla, ponieważ to on definiuje budżet pamięci, w ramach którego działają pozostałe techniki. Dobiera się go indywidualnie dla każdej rozdzielczości wejściowej tak, aby mieścił się w budżecie pamięci przy możliwie najmniejszej redundancji obliczeń. W kolejnym kroku mechanizm selekcji rzadkiej jest zastępowany przez *SpargAttention* i włączana jest kwantyzacja wag. Zabieg ten jednocześnie skraca czas inferencji i zmniejsza szczytowe zużycie pamięci przy minimalnym wpływie na jakość rekonstrukcji. Kwantyzację aktywacji należy natomiast traktować jako rezerwę, którą warto włączyć dopiero wtedy, gdy uzyskany wcześniej zapas pamięci okaże się niewystarczający.

9.3 Ograniczenia i kierunki rozwoju

Wyniki uzyskano wyłącznie na jednej architekturze GPU, dlatego wniosków nie można bezpośrednio przenieść na inne generacje sprzętu. W szczególności układy obsługujące format FP8 otwierają ścieżkę optymalizacji, która w ramach tej pracy była niedostępna. Ze względu na czas potrzebny do przetworzenia obu zbiorów testowych jakość rekonstrukcji oceniano na materiale wygenerowanym na akceleratorze A100, a nie na karcie docelowej. Ponadto pomiary objęły dwie rozdzielczości wejściowe, obie niższe od rozdzielczości typowych nagrań, przez co uzyskane wartości liczbowe należy odnosić wyłącznie do badanego zakresu.

Warto rozważyć trzy główne kierunki dalszych prac. Pierwszym z nich jest szczegółowa ocena artefaktów powstających na granicach kafli, zarówno przestrzennych, jak i czasowych. Wykorzystane w badaniach metryki obliczane są dla całych klatek, przez co mogą nie wychwytywać lokalnych nieciągłości obrazu. Drugi kierunek to opracowanie metody automatycznego doboru rozmiaru kafla przestrzennego dla zadanego wejścia. Taki mechanizm minimalizowałby redundancję obliczeń i jest możliwy do zrealizowania na podstawie samej geometrii podziału, jeszcze przed właściwym przetwarzaniem. Trzeci obszar badawczy wynika z wniosku, że to kafelkowanie przestrzenne odpowiada za niemal cały spadek jakości. Ponieważ model bazowy był trenowany na pełnych klatkach, jego dostrojenie na danych kafelkowanych prawdopodobnie pozwoliłoby zniwelować te straty. Działanie to wykracza jednak poza zakres zdefiniowany w podrozdziale 1.2, a dodatkową barierą jest fakt, że autorzy modelu FlashVSR opisali jedynie swoją wieloetapową procedurę trenowania, nie publikując niezbędnego do niej kodu.

Bibliografia

- [1] J. Zhuang *et al.*, „FlashVSR: Towards Real-Time Diffusion-Based Streaming Video Super-Resolution”, *ArXiv*, 2025.
- [2] A. Vaswani *et al.*, „Attention Is All You Need”. [Online]. Dostępne na: <https://arxiv.org/abs/1706.03762>
- [3] A. A. Baniya, T.-K. Lee, P. W. Eklund, i S. Aryal, „A Survey of Deep Learning Video Super-Resolution”, *IEEE Transactions on Emerging Topics in Computational Intelligence*, t. 8, nr 4, s. 2655–2676, sie. 2024, doi: 10.1109/tetci.2024.3398015.
- [4] H. Liu *et al.*, „Video Super Resolution Based on Deep Learning: A Comprehensive Survey”. [Online]. Dostępne na: <https://arxiv.org/abs/2007.12928>
- [5] K. C. K. Chan, S. Zhou, X. Xu, i C. C. Loy, „Investigating Tradeoffs in Real-World Video Super-Resolution”. [Online]. Dostępne na: <https://arxiv.org/abs/2111.12704>
- [6] L. Yang *et al.*, „Diffusion Models: A Comprehensive Survey of Methods and Applications”. [Online]. Dostępne na: <https://arxiv.org/abs/2209.00796>
- [7] J. Ho, A. Jain, i P. Abbeel, „Denoising Diffusion Probabilistic Models”. [Online]. Dostępne na: <https://arxiv.org/abs/2006.11239>
- [8] A. Dosovitskiy *et al.*, „An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”. [Online]. Dostępne na: <https://arxiv.org/abs/2010.11929>
- [9] W. Peebles i S. Xie, „Scalable Diffusion Models with Transformers”. [Online]. Dostępne na: <https://arxiv.org/abs/2212.09748>
- [10] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, i C. Ré, „FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness”. [Online]. Dostępne na: <https://arxiv.org/abs/2205.14135>
- [11] J. Zhang, J. Wei, H. Huang, P. Zhang, J. Zhu, i J. Chen, „SageAttention: Accurate 8-Bit Attention for Plug-and-play Inference Acceleration”. [Online]. Dostępne na: <https://arxiv.org/abs/2410.02367>
- [12] J. Zhang, H. Huang, P. Zhang, J. Wei, J. Zhu, i J. Chen, „SageAttention2: Efficient Attention with Thorough Outlier Smoothing and Per-thread INT4 Quantization”. [Online]. Dostępne na: <https://arxiv.org/abs/2411.10958>
- [13] J. Zhang *et al.*, „SpargeAttention: Accurate and Training-free Sparse Attention Accelerating Any Model Inference”. [Online]. Dostępne na: <https://arxiv.org/abs/2502.18137>
- [14] H. Wu, P. Judd, X. Zhang, M. Isaev, i P. Micikevicius, „Integer Quantization for Deep Learning Inference: Principles and Empirical Evaluation”. [Online]. Dostępne na: <https://arxiv.org/abs/2004.09602>
- [15] F. A. Fardo, V. H. Conforto, F. C. de Oliveira, i P. S. Rodrigues, „A Formal Evaluation of PSNR as Quality Measurement Parameter for Image Segmentation Algorithms”. [Online]. Dostępne na: <https://arxiv.org/abs/1605.07116>

-
- [16] J. Nilsson i T. Akenine-Möller, „Understanding SSIM”. [Online]. Dostępne na: <https://arxiv.org/abs/2006.13846>
 - [17] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, i O. Wang, „The Unreasonable Effectiveness of Deep Features as a Perceptual Metric”. [Online]. Dostępne na: <https://arxiv.org/abs/1801.03924>
 - [18] A. Zvezdakova, D. Kulikov, D. Kondranin, i D. Vatolin, „Barriers towards no-reference metrics application to compressed video quality analysis: on the example of no-reference metric NIQE”. [Online]. Dostępne na: <https://arxiv.org/abs/1907.03842>
 - [19] J. Ke, Q. Wang, Y. Wang, P. Milanfar, i F. Yang, „MUSIQ: Multi-scale Image Quality Transformer”. [Online]. Dostępne na: <https://arxiv.org/abs/2108.05997>
 - [20] J. Wang, K. C. K. Chan, i C. C. Loy, „Exploring CLIP for Assessing the Look and Feel of Images”. [Online]. Dostępne na: <https://arxiv.org/abs/2207.12396>
 - [21] H. Wu *et al.*, „Exploring Video Quality Assessment on User Generated Contents from Aesthetic and Technical Perspectives”. [Online]. Dostępne na: <https://arxiv.org/abs/2211.04894>
 - [22] X. Wang, K. C. K. Chan, K. Yu, C. Dong, i C. C. Loy, „EDVR: Video Restoration with Enhanced Deformable Convolutional Networks”. [Online]. Dostępne na: <https://arxiv.org/abs/1905.02716>
 - [23] K. C. K. Chan, X. Wang, K. Yu, C. Dong, i C. C. Loy, „BasicVSR: The Search for Essential Components in Video Super-Resolution and Beyond”. [Online]. Dostępne na: <https://arxiv.org/abs/2012.02181>
 - [24] J. Cao, Y. Li, K. Zhang, i L. V. Gool, „Video Super-Resolution Transformer”. [Online]. Dostępne na: <https://arxiv.org/abs/2106.06847>
 - [25] J. Liang *et al.*, „VRT: A Video Restoration Transformer”. [Online]. Dostępne na: <https://arxiv.org/abs/2201.12288>
 - [26] S. Zhou, P. Yang, J. Wang, Y. Luo, i C. C. Loy, „Upscale-A-Video: Temporal-Consistent Diffusion Model for Real-World Video Super-Resolution”. [Online]. Dostępne na: <https://arxiv.org/abs/2312.06640>
 - [27] Z. Chen *et al.*, „DOVE: Efficient One-Step Diffusion Model for Real-World Video Super-Resolution”. [Online]. Dostępne na: <https://arxiv.org/abs/2505.16239>
 - [28] J. Wang *et al.*, „SeedVR2: One-Step Video Restoration via Diffusion Adversarial Post-Training”. [Online]. Dostępne na: <https://arxiv.org/abs/2506.05301>
 - [29] NVIDIA Corporation, „NVIDIA Ampere GA102 Architecture Whitepaper”, technical report, 2020. [Online]. Dostępne na: <https://www.nvidia.com/content/PDF/nvidia-ampere-ga-102-gpu-architecture-whitepaper-v2.pdf>
 - [30] PyTorch Team, „PyTorch documentation”.
 - [31] PyTorch Team, „torchao: PyTorch-Native Training-to-Serving Model Optimization”.
 - [32] OpenImagingLab, „FlashVSR”.

Spis rysunków

Rysunek 1 Schemat przetwarzania w modelu FlashVSR	12
Rysunek 2 Przepływ przetwarzania nagrania w zaimplementowanym potoku	18
Rysunek 3 Interfejs aplikacji pokazowej	22
Rysunek 4 Porównanie jakościowe dla zbioru VideoLQ, klip 6, klatka 13.	32
Rysunek 5 Porównanie jakościowe dla zbioru VideoLQ, klip 8, klatka 0.	32

Spis tabel

Tabela 1 Porównanie klas metod superrozdzielczości wideo	14
Tabela 2 Zestawienie technik wybranych do implementacji	16
Tabela 3 Zestawienie przeprowadzonych eksperymentów	26
Tabela 4 Czas inferencji i szczytowe zużycie pamięci dla dwunastu kombinacji mechanizmu uwagi i trybu kwantyzacji	27
Tabela 5 Zmiany względem konfiguracji odniesienia	27
Tabela 6 Czas inferencji i szczytowe zużycie pamięci w funkcji rozmiaru kafla	28
Tabela 7 Zapotrzebowanie konfiguracji bazowej	29
Tabela 8 Przesunięcie granicy wykonalności	30
Tabela 9 Metryki jakości dla zbioru YouHQ40	30
Tabela 10 Zmiana metryk względem konfiguracji poprzedniej, zbiór YouHQ40	31
Tabela 11 Metryki jakości dla różnych rozmiarów kafla, zbiór YouHQ40.	31
Tabela 12 Metryki bezreferencyjne dla zbioru VideoLQ	31
Tabela 13 Zmiana metryk względem konfiguracji poprzedniej, zbiór VideoLQ	32